

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ПО ПРОГРАММЕ БАКАЛАВРИАТА**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Разработка программно-информационных систем»

Бизнес-проект «Онлайн-площадка для сдачи вещей в аренду».

Разработка серверной части

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

(подпись, дата)

В. А. Ермаков

(инициалы, фамилия)

Группа ПО-216

Руководитель ВКР

(подпись, дата)

Е. А. Петрик

(инициалы, фамилия)

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

(подпись, дата)

А. В. Малышев

(инициалы, фамилия)

Курск 2026 г.

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

(подпись, инициалы, фамилия)

« ____ » _____ 20 ____ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ БАКАЛАВРИАТА

Студента Ермакова В.А., шифр 22-06-0116, группа ПО-216

1. Тема «Бизнес-проект «Онлайн-площадка для сдачи вещей в аренду». Разработка серверной части» утверждена приказом ректора ЮЗГУ от «03» апреля 2026 г. № 1584-с.

2. Срок предоставления работы к защите «09» июня 2026 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

1) провести анализ предметной области аренды вещей и существующих онлайн-площадок;

2) разработать концептуальную модель серверной части онлайн-площадки, определить основные сущности и связи между ними;

3) спроектировать базу данных и архитектуру программного интерфейса (REST API) для взаимодействия с клиентской частью;

4) разработать и протестировать серверную часть онлайн-площадки, реализующую хранение данных, бизнес-логику и обработку запросов.

3.2. Входные данные и требуемые результаты для программы:

1) Входными данными для программной системы являются: регистрационные данные пользователей, профильные данные пользователей, данные

справочников категорий вещей, информация о вещах, выставляемых на аренду, параметры бронирования, текстовые сообщения пользователей в чатах, оценки и отзывы по завершенным сделкам, поисковые запросы и параметры фильтрации объявлений.

2) Выходными данными для программной системы являются: сформированные объявления о вещах, подтвержденные бронирования с автоматически рассчитанной общей стоимостью, история аренды для каждого пользователя, статусы бронирований, уведомления об изменениях статусов, сообщения чата, привязанные к сделкам, рейтинги пользователей, обновляемые на основе отзывов, результаты поиска и фильтрации объявлений, а также статистические отчеты по популярным категориям, активности пользователей и завершенным арендам.

4. Содержание работы (по разделам):

4.1. Введение.

4.1. Анализ предметной области.

4.2. Техническое задание: основание для разработки, назначение разработки, требования к программной системе, требования к оформлению документации.

4.3. Технический проект: общие сведения о программной системе, проект данных программной системы, проектирование архитектуры программной системы, проектирование пользовательского интерфейса программной системы.

4.4. Рабочий проект: спецификация компонентов и классов программной системы, тестирование программной системы.

4.5. Заключение.

4.6. Список использованных источников.

5. Перечень графического материала:

Руководитель ВКР

(подпись, дата)

Е. А. Петрик

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

В. А. Ермаков

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 0 страницам. Работа содержит 0 иллюстраций, 0 таблиц, 0 библиографических источников и 0 листов графического материала. Количество приложений – 2. Графический материал представлен в приложении А. Фрагменты исходного кода представлены в приложении Б.

Перечень ключевых слов: онлайн-площадка, аренда вещей, серверная часть, бэкенд, Python, Django, Django REST Framework, PostgreSQL, REST API, база данных, сервер, Ubuntu Server, Nginx, Gunicorn, аутентификация, авторизация, защита информации, DDoS-защита, модель данных, миграция, сущность, веб-сервер, обратный прокси, SSH, API-endpoint.

Объектом разработки является серверная часть онлайн-площадки для аренды вещей предназначенная для хранения, обработки и предоставления данных, бизнес-логику приложения, а также взаимодействие с клиентской частью (и модулем коммуникации через REST API.

Целью выпускной квалификационной работы является разработка и развертывание серверной части онлайн-площадки для аренды вещей, обеспечивающей надежное хранение данных, эффективную бизнес-логику, безопасное взаимодействие с клиентами и защиту от сетевых атак.

В процессе разработки серверной части онлайн-площадки для аренды вещей были проведены анализ предметной области и существующих решений, спроектирована база данных, выделены основные сущности (пользователи, категории, объявления, бронирования, отзывы, сообщения) и установлены связи между ними, разработаны классы и методы, реализующие бизнес-логику приложения и обеспечивающие корректную обработку запросов, создан программный интерфейс для взаимодействия с клиентской частью и модулем коммуникации, реализована система аутентификации и разграничения прав доступа, развернута и настроена серверная инфраструктура, обеспечена защита от сетевых атак, организовано удаленное администрирование, проведена интеграция с компонентами, разработанными другими участниками команды.

ABSTRACT

The volume of work is 0 pages. The work contains 0 illustrations, 0 tables, 0 bibliographic sources and 0 sheets of graphic material. The number of applications is 2. The graphic material is presented in annex A. The layout of the site, including the connection of components, is presented in annex B.

The list of keywords: online marketplace, rental of things, server side, backend, Python, Django, Django REST Framework, PostgreSQL, REST API, database, server, Ubuntu Server, Nginx, Gunicorn, authentication, authorization, information protection, DDoS protection, data model, migration, entity, web-server, reverse proxy, SSH, API endpoint.

The object of development is the server part of an online marketplace for renting things designed for storing, processing and providing data, the business logic of the application, as well as interaction with the client part (and the communication module via the REST API).

The purpose of the final qualifying work is to develop and deploy the back-end of an online marketplace for renting things, providing reliable data storage, efficient business logic, secure interaction with customers and protection from network attacks.

During the development of the server side of the online rental platform, an analysis of the subject area and existing solutions was carried out, a database was designed, the main entities (users, categories, ads, reservations, reviews, messages) were identified and links between them were established, classes and methods were developed that implement the business logic of the application and ensure correct processing requests, a software interface has been created for interacting with the client side and the communication module, and an authentication and access rights management system has been implemented., The server infrastructure has been deployed and configured, protection against network attacks has been provided, remote administration has been organized, and integration with components developed by other team members has been carried out.

The developed website was successfully implemented in the company.

СОДЕРЖАНИЕ

РЕЗЮМЕ СТАРТАП-ПРОЕКТА	9
ВВЕДЕНИЕ	12
1 Анализ предметной области	14
1.1 Понятие и принципы онлайн-аренды вещей	14
1.2 Классификация онлайн-площадок для аренды	16
1.2.1 По составу предлагаемых товарных категорий	17
1.2.2 По модели участия платформы в транзакции	17
1.2.3 По типу арендодателей	18
1.3 Основные бизнес-процессы системы аренды	18
1.3.1 Регистрация и верификация пользователей	18
1.3.2 Публикация и поиск предложений	19
1.3.3 Бронирование и совершение сделки	19
1.3.4 Система рейтингов и отзывов	20
1.4 Компоненты серверной части программно-информационной системы	20
1.4.1 Модуль управления пользователями	20
1.4.2 Модуль каталога и поиска	21
1.4.3 Модуль бронирования и управления доступностью	21
1.4.4 Модуль аналитики и отчётности	21
1.5 Правовое регулирование онлайн-платформ аренды в Российской Федерации	21
1.6 Анализ существующих решений и аналогов	23
2 Техническое задание	25
2.1 Основание для разработки	25
2.2 Цель и назначение разработки	25
2.3 Требования к серверной части веб-приложения	25
2.3.1 Функциональные требования к серверной части	25
2.3.2 Требования к программному интерфейсу	28
2.4 Моделирование вариантов использования	29

2.4.1	Регистрация с подтверждением email	33
2.4.2	Аутентификация и JWT-токен	34
2.4.3	Восстановление пароля	35
2.4.4	Просмотр профиля	35
2.4.5	Обновление профиля	36
2.4.6	Автозаполнение реквизитов по ИНН	36
2.4.7	Просмотр каталога объявлений	37
2.4.8	Фильтрация и поиск объявлений	37
2.4.9	Просмотр карточки объявления	38
2.4.10	Создание объявления	38
2.4.11	Редактирование объявления	39
2.4.12	Удаление объявления	39
2.4.13	Загрузка изображений	40
2.4.14	Удаление изображений	40
2.4.15	Модерация объявлений	41
2.4.16	Создание категории	41
2.4.17	Удаление категории	42
2.4.18	Блокировка пользователя	42
2.4.19	Разблокировка пользователя	43
2.4.20	Блокировка объявления	43
2.4.21	Разблокировка объявления	44
2.4.22	Создание бронирования	44
2.4.23	Просмотр бронирований	45
2.4.24	Подтверждение бронирования	45
2.4.25	Отмена бронирования	46
2.4.26	Завершение аренды	46
2.4.27	Оплата через СБП	47
2.4.28	Подтверждение получения товара	47
2.4.29	Подтверждение возврата товара	48
2.4.30	Возврат залога	49
2.4.31	Возврат средств через СБП	49

2.4.32 Создание отзыва	50
2.5 Требования к оформлению документации	50
3 Технический проект	51
3.1 Архитектура онлайн-площадки для сдачи вещей в аренду	51
3.2 Обоснование выбора технологии проектирования	52
3.2.1 Язык программирования Python	52
3.2.2 Веб-фреймворк Django	52
3.2.3 Система управления базами данных PostgreSQL	52
3.2.4 Система контейнеризации Docker	53
3.2.5 Сервер-обработчик Nginx	53
3.2.6 WSGI-сервер Gunicorn	53
3.2.7 ASGI-сервер Daphne	53
3.3 Архитектура серверного приложения	54
3.3.1 Структурная декомпозиция и порядок запуска	54
3.3.2 Диаграмма компонентов	55
3.3.3 Маршрутизация API-endpoints	57
3.4 Модель данных	81
3.4.1 Нормализация схемы данных	82
3.4.2 Описание структуры таблиц	82
3.5 Реализация бизнес-логики	89
3.5.1 Управление товарами	89
3.5.2 Бронирование и проверка конфликтов дат	89
3.5.3 Платёжный модуль	90
3.5.4 Система отзывов и рейтинга	90
3.6 Информационная безопасность	90
3.6.1 Профиль угроз	90
3.6.2 Меры защиты	91
3.7 Эксплуатация и развёртывание	91
4 Рабочий проект	92
4.1 Модуль users	92
4.1.1 Класс SendEmailVerificationCodeView	92

4.1.2	Класс RegisterView	92
4.1.3	Класс FillByInnView	93
4.1.4	Класс ProfileView	93
4.1.5	Класс ChangePasswordView	94
4.1.6	Класс PasswordResetRequestView	94
4.1.7	Класс PasswordResetConfirmView	95
4.2	Модуль items	95
4.2.1	Класс CategoryViewSet	96
4.2.2	Класс ItemViewSet	96
4.2.3	Класс ItemImageUploadView	97
4.2.4	Класс ItemImageDeleteView	98
4.2.5	Класс ItemModerationRequestViewSet	98
4.3	Модуль bookings	99
4.3.1	Класс BookingViewSet	99
4.4	Модуль arenda_site	101
4.4.1	settings.py	102
4.4.2	urls.py	104
4.5	Интеграционное тестирование API	105
	ЗАКЛЮЧЕНИЕ	109
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	110
	ПРИЛОЖЕНИЕ А Представление графического материала	117
	ПРИЛОЖЕНИЕ Б Фрагменты исходного кода программы	128
	На отдельных листах (CD-RW в прикрепленном конверте)	139
	Сведения о ВКРБ (Графический материал / Сведения о ВКРБ.png)	Лист 1
	Цель и задачи разработки (Графический материал / Цель и задачи разработки.png)	Лист 2
	Сравнение существующих онлайн-площадок для аренды вещей (Графический материал / Сравнение существующих онлайн-площадок для аренды вещей.png)	Лист 3
	Компоненты онлайн-площадки для аренды вещей (Графический материал / Компоненты онлайн-площадки для аренды вещей.png)	Лист 4

Диаграмма прецедентов. Часть 1 (Графический материал / Диаграмма прецедентов. Часть 1.png)	Лист 5
Диаграмма прецедентов. Часть 2 (Графический материал / Диаграмма прецедентов. Часть 2.png)	Лист 6
Диаграмма прецедентов. Часть 3 (Графический материал / Диаграмма прецедентов. Часть 3.png)	Лист 7
Диаграмма компонентов (Графический материал / Диаграмма компонентов.png)	Лист 8
Модель данных (Графический материал / Модель данных.png)	Лист 9
Заключение (Графический материал / Заключение.png)	Лист 10

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

БД – база данных.

ИС – информационная система.

ИТ – информационные технологии.

ПО – программное обеспечение.

РП – рабочий проект.

ТЗ – техническое задание.

ТП – технический проект.

ПО – программное обеспечение.

СУБД – система управления базами данных.

API (Application Programming Interface) – программный интерфейс приложения, набор правил и инструментов для взаимодействия программных компонентов.

HTTP (HyperText Transfer Protocol) – протокол передачи гипертекста.

HTTPS (HyperText Transfer Protocol Secure) – защищенный протокол передачи гипертекста.

SQL (Structured Query Language) – язык структурированных запросов к базам данных.

REST (Representational State Transfer) — архитектурный стиль взаимодействия компонентов распределённого приложения через HTTP.

JWT (JSON Web Token) — формат токена, используемый для аутентификации и авторизации пользователей.

JSON (JavaScript Object Notation) — текстовый формат обмена данными, применяемый для передачи информации между клиентской и серверной частями системы.

CRUD (Create, Read, Update, Delete) — основные операции по работе с данными: создание, чтение, обновление и удаление.

PDF (Portable Document Format) — формат электронных документов, используемый для генерации договоров аренды.

РЕЗЮМЕ СТАРТАП-ПРОЕКТА

Название: «Онлайн-площадка для сдачи вещей в аренду». Проект представляет собой веб-приложение, предназначенное для автоматизации процессов аренды различных товаров между частными лицами или компаниями. Разработанная система должна цифровизировать такие задачи как просмотр каталога доступных вещей, бронирование товаров на определенный срок, расчет стоимости аренды, обработка заявок, а также взаимодействие между арендодателями и арендаторами. В ходе работы были проанализированы современные аналоги и сервисы совместного потребления (шеринг-платформы), и выявлены их отрицательные стороны:

1. Интерфейс. Многие существующие платформы перегружены элементами, рекламой и второстепенными функциями, что затрудняет быстрый поиск нужного товара и оформление аренды для новых пользователей.

2. Высокие комиссии и скрытые платежи за аренду товара или услуги. Ряд популярных сервисов взимают значительную комиссию как с арендодателей, так и с арендаторов, либо требуют оформления платной подписки для доступа к базовым возможностям, что снижает экономическую выгоду от использования платформы.

3. Отсутствие гибкости в настройке условий аренды вещей или услуг. Во многих системах отсутствует возможность гибко настроить условия аренды для конкретного товара или услуги: посуточная/почасовая аренда, размер залога, стоимость доставки, что ограничивает владельцев вещей.

4. Неудобная система коммуникации между арендатором и арендодателем. Встроенные чаты или формы связи часто работают с задержками, либо требуют перехода в сторонние мессенджеры, что создает неудобства и риски для безопасности сделки.

Создаваемое приложение для аренды вещей позволит избежать вышеперечисленные недостатки без негативного влияния на процесс работы и скорость выполнения поставленных задач, предлагая сбалансированное решение с понятным интерфейсом и прозрачными условиями.

Актуальность проекта связана с ростом популярности модели совместного потребления (шеринг-экономики) в России и мире. Большое количество людей предпочитают арендовать вещи для разового или краткосрочного использования вместо их покупки, однако существующие площадки не всегда отвечают потребностям пользователей в простоте, надежности и доступности.

Целью является разработка полнофункционального веб-приложения для аренды вещей, обеспечивающего удобный поиск, бронирование и сдачу товаров в аренду.

Отличительные особенности разрабатываемого приложения по сравнению с уже существующими решениями:

1. Интерфейс с продуманной структурой каталога и фильтрацией, позволяющей быстро подобрать вещь по категории, цене, сроку аренды.

2. Прозрачная система формирования стоимости с автоматическим расчетом итоговой цены в зависимости от количества дней аренды, с возможностью учета скидок и залога.

3. Реализация личного кабинета для арендодателей и арендаторов с разделением возможностей: управление объявлениями, просмотр истории бронирований, отслеживание статусов заказов.

4. Интеграция с серверным API для выполнения основных операций (получение списка товаров, отправка заявок на аренду, добавление в корзину) без перезагрузки страницы, что обеспечивает высокую скорость работы и комфорт пользователя.

5. Адаптивная верстка, обеспечивающая корректное отображение и удобство использования сайта на всех типах устройств: персональных компьютерах, планшетах и смартфонах.

В разработанном приложении для аренды вещей реализованы такие функции как: просмотр каталога товаров с фильтрацией, регистрация и авторизация пользователей, добавление новых объявлений о сдаче вещей в аренду, оформление заказа с выбором дат аренды, корзина аренды, личный каби-

нет с историей операций, а также административная панель для модерации объявлений и управления пользователями.

Идея создания проекта появилась в сентябре 2025 года, активная разработка была начата в феврале 2026 года. С апреля 2026 года производилось тестирование системы путем моделирования различных пользовательских сценариев: размещение объявлений, поиск вещей, оформление бронирований. Проект планируется закончить к маю 2026 года. Также будет осуществлена регистрация программного продукта для защиты авторских прав и обеспечения исключительности прав.

Бизнес-цели и перспективы дальнейшего развития проекта:

1. Достижение 2 500 успешных завершённых сделок (аренд) в течение первых 12 месяцев работы площадки.
2. Обеспечение доли повторных аренд (пользователь арендует второй раз и более) на уровне не менее 30% от общего числа активных клиентов к концу 1-го года.
3. Снижение доли спорных сделок и возвратов залогов до менее 5% от общего числа аренд путём внедрения системы верификации и страховки.
4. Достижение среднего чека одной аренды на уровне не менее 1 500 рублей.
5. Обеспечение времени отклика арендодателя на запрос арендатора не более 2 часов в 90% случаев (для удержания пользователей).

ВВЕДЕНИЕ

Современные интернет-технологии активно развиваются и проникают во все сферы жизни. Одним из ярких проявлений этого процесса стало появление разных площадок для торговли вещами, в рамках которой люди получают возможность приобретать вещи как в собственность, так и арендовать их у других пользователей. Онлайн-площадки для аренды вещей позволяют эффективно использовать временно не востребуемые предметы, экономя ресурсы и материальные средства. Интенсивность развития данного направления не имеет аналогов — от первых досок объявлений до современных платформ с автоматизированным бронированием, системой рейтингов и встроенными чатами.

При разработке онлайн-площадки все стадии реализации проекта — от проектирования базы данных до развертывания серверной части — находятся в единой технологической цепи. Серверная часть является ядром любой современной веб-платформы, обеспечивая хранение данных, выполнение бизнес-логики и взаимодействие с клиентскими приложениями через программные интерфейсы.

Современные компании, видя, как развиваются информационные технологии, стремятся использовать их для развития своего бизнеса. Онлайн-площадка позволяет не только заявить о себе, но и предоставить пользователям удобный сервис для размещения объявлений, поиска нужных вещей, бронирования и общения. Качественно реализованная серверная часть обеспечивает надежность, безопасность и масштабируемость всего проекта.

Цель настоящей работы — разработка серверной части онлайн-площадки для аренды вещей, обеспечивающей хранение данных, бизнес-логику и взаимодействие с клиентской частью. Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и существующих решений в сфере аренды вещей;

- спроектировать базу данных, выделить основные сущности и установить связи между ними;
- разработать классы и методы, реализующие бизнес-логику приложения;
- создать программный интерфейс для взаимодействия с клиентской частью и модулем коммуникации.
- развернуть и настроить серверную инфраструктуру

Структура и объем работы. Отчет состоит из введения, 4 разделов основной части, заключения, списка использованных источников, 2 приложений. Текст выпускной квалификационной работы равен 0 страницам.

Во введении сформулирована цель работы, поставлены задачи разработки, описана структура работы, приведено краткое содержание каждого из разделов.

В первом разделе проводится анализ предметной области аренды вещей, рассматриваются существующие онлайн-площадки, определяются функциональные требования к разрабатываемой системе.

Во втором разделе формулируются требования к серверной части, включая требования к базе данных, программному интерфейсу, производительности и безопасности.

В третьем разделе на стадии технического проектирования представлены проектные решения для серверной части.

В четвертом разделе описывается реализация серверной части, приводится описание разработанных классов и методов, рассматриваются вопросы развертывания и настройки серверной инфраструктуры, проводится тестирование разработанной системы.

В заключении излагаются основные результаты работы, полученные в ходе разработки.

В приложении А представлен графический материал. В приложении Б представлены фрагменты исходного кода.

1 Анализ предметной области

1.1 Понятие и принципы онлайн-аренды вещей

Электронная коммерция представляет собой совокупность коммерческих и финансовых операций, осуществляемых посредством глобальных компьютерных сетей, а также комплекс сопутствующих бизнес-процессов, обеспечивающих их проведение. В рамках данной сферы онлайн-аренда материальных объектов сформировалась как самостоятельный сегмент, характеризующийся временной передачей права пользования вещью через интернет-платформу на возмездной основе без перехода права собственности к пользователю.

Концептуальной основой онлайн-аренды служит модель совместного потребления (sharing economy), предполагающая эффективное распределение недозагруженных материальных ресурсов посредством цифровых посредников. С инженерной точки зрения реализация данной концепции требует создания многоуровневой программно-информационной системы, центральным звеном которой выступает серверная часть, несущая ответственность за обработку бизнес-логики, персистентное хранение данных, управление транзакциями и обеспечение информационной безопасности.

Принята следующая классификация субъектов электронной коммерции по типу участников:

- C2C (Consumer-to-Consumer) — взаимодействие между конечными потребителями;
- B2C (Business-to-Consumer) — взаимодействие коммерческой организации с конечным потребителем;
- B2B (Business-to-Business) — взаимодействие хозяйствующих субъектов между собой;
- B2G (Business-to-Government) — взаимодействие коммерческих структур с органами государственного управления;
- P2P (Peer-to-Peer) — децентрализованное взаимодействие равноправных участников без выделенного посредника.

В сегменте аренды вещей доминируют модели C2C и B2C, что непосредственно определяет требования к ролевой модели и разграничению прав доступа в проектируемой серверной части.

Онлайн-бронирование представляет собой процесс резервирования товара или услуги в интерактивном режиме реального времени посредством сети Интернет. Данный механизм широко применяется при резервировании номерного фонда гостиниц, приобретении транспортных билетов, бронировании столиков в заведениях общественного питания и учреждениях культуры. Применительно к аренде вещей онлайн-бронирование приобретает дополнительную специфику: серверная часть системы обязана обеспечивать управление временной доступностью каждого объекта, предотвращение конкурентных резервирований, автоматизированный расчёт стоимости, а также управление финансовыми обязательствами сторон в течение всего жизненного цикла сделки.

Использование онлайн-платформ аренды обеспечивает следующие преимущества для участников, каждое из которых транслируется в конкретные требования к серверным компонентам системы:

1. Экономическая эффективность. Арендаторы получают доступ к дорогостоящим или редко используемым объектам без необходимости их приобретения в собственность; арендодатели извлекают доход от временно незадействованного имущества. Серверная часть обеспечивает автоматизированный расчёт итоговой стоимости с учётом комиссионного вознаграждения платформы.

2. Доступность и масштабируемость. Поиск предложений, оформление бронирования и проведение оплаты доступны в непрерывном режиме из любой географической точки при наличии подключения к сети Интернет. Архитектура серверной части должна обеспечивать стабильную обработку запросов при значительных пиковых нагрузках.

3. Прозрачность и верифицируемость сделок. Встроенная система рейтингов и отзывов формирует репутационный профиль каждого участника, снижая информационную асимметрию при заключении сделок. Серверная

часть обеспечивает хранение, агрегирование рейтинговых данных и их учёт в алгоритмах ранжирования объявлений.

Наряду с преимуществами функционирование платформы сопряжено с рисками, которые подлежат учёту на стадии проектирования серверной части:

1. Несоответствие объекта аренды заявленным характеристикам. Ввиду отсутствия возможности физической инспекции предмета до завершения бронирования существует вероятность расхождения между реальным состоянием объекта и его описанием в карточке объявления. Для снижения данного риска серверная часть должна реализовывать валидацию и модерацию загружаемых медиаматериалов.

2. Риски, связанные с передачей и возвратом имущества. Организация логистики влечёт дополнительные издержки, а в процессе доставки или эксплуатации возможны утрата либо повреждение предмета аренды. Серверная часть фиксирует статусы передачи и приёмки, формируя доказательную базу для урегулирования споров.

3. Юридические и финансовые разногласия. При отсутствии встроенных механизмов обеспечения сделки урегулирование споров о возмещении ущерба или возврате депозита существенно затрудняется. Реализация на стороне сервера механизма условного удержания денежных средств (escrow) и автоматической генерации договора аренды снижает данные риски.

4. Угрозы информационной безопасности. Серверная часть является основным объектом атак, направленных на хищение персональных данных пользователей и осуществление несанкционированных финансовых операций. Обеспечение безопасности достигается посредством применения JWT-аутентификации, шифрования канала передачи данных по протоколу HTTPS, а также использования сертифицированных платёжных шлюзов.

1.2 Классификация онлайн-площадок для аренды

Современный рынок сервисов онлайн-аренды характеризуется значительной неоднородностью предлагаемых решений. Систематизация суще-

ствующих платформ по ключевым признакам позволяет обоснованно определить место проектируемой системы в данном сегменте и сформулировать архитектурные требования к её серверной части.

1.2.1 По составу предлагаемых товарных категорий

По степени предметной специализации выделяются два типа платформ: универсальные и нишевые (специализированные).

Универсальные платформы агрегируют объявления по широкому спектру категорий: электроника, строительный инструмент, спортивный инвентарь, предметы одежды и т.д. Конкурентным преимуществом таких систем служат масштаб аудитории и объём предложений. Вместе с тем в части серверного проектирования универсальность обуславливает необходимость поддержки полиморфной структуры атрибутов объявлений, что усложняет схему базы данных и логику серверной валидации.

Нишевые платформы сосредоточены на одной или нескольких смежных товарных категориях. Данное ограничение позволяет применить фиксированную реляционную схему для хранения атрибутов объявлений, упростить серверную валидацию и обеспечить более высокую релевантность поисковой выдачи. Проектируемая система тяготеет к универсальной модели, что определяет соответствующие требования к архитектуре модуля каталога.

1.2.2 По модели участия платформы в транзакции

В зависимости от степени вовлечённости платформы в процесс заключения и сопровождения сделки различают три функциональные модели.

Модель классифайда предполагает, что платформа выполняет исключительно функцию доски объявлений: публикует предложения арендодателей, тогда как условия оплаты и передачи имущества участники согласовывают самостоятельно за пределами системы. Серверная часть в данном случае ограничивается функциями хранения объявлений и базовой аутентификации пользователей.

Модель полного сопровождения сделки предусматривает, что платформа принимает на себя обработку и временное удержание платежей, генерацию юридически значимых документов, информирование сторон о смене статусов и участие в урегулировании споров, взимая комиссионное вознаграждение с каждой транзакции. Именно данная модель положена в основу проектируемой системы. Её реализация предъявляет наиболее высокие требования к серверной архитектуре: необходимы самостоятельные функциональные модули для обработки платежей, генерации договоров, WebSocket-коммуникации и управления уведомлениями.

Подписочная модель предполагает внесение пользователем периодического платежа за право неограниченного доступа к аренде вещей из заранее сформированного каталога. Данная модель наиболее востребована в категориях детских товаров и одежды.

1.2.3 По типу арендодателей

По составу участников сделки на стороне предложения выделяются две основные модели. Модель C2C характеризуется тем, что арендодателями выступают физические лица, предоставляющие во временное пользование объекты личной собственности. Модель B2C предполагает участие коммерческих организаций, осуществляющих профессиональную деятельность по прокату имущества и использующих платформу в качестве дополнительного канала привлечения клиентов. Поскольку проектируемая система ориентирована на поддержку обеих моделей, серверная часть должна реализовывать гибкое ролевое разграничение с возможностью совмещения ролей арендодателя и арендатора в рамках одной учётной записи.

1.3 Основные бизнес-процессы системы аренды

1.3.1 Регистрация и верификация пользователей

Обеспечение взаимного доверия участников платформы и снижение рисков мошенничества и нецелевого использования имущества достигается

посредством многоуровневой процедуры регистрации и верификации. Серверная часть реализует последовательность этапов: создание учётной записи с подтверждением адреса электронной почты через одноразовый верификационный код, опциональную привязку платёжного инструмента, а в установленных случаях — верификацию личности посредством интеграции с государственными информационными сервисами. Верификационные данные сохраняются в базе данных в зашифрованном виде с жёстким разграничением прав доступа. Архитектура ролевой модели допускает совмещение функций арендодателя и арендатора в рамках единой учётной записи.

1.3.2 Публикация и поиск предложений

Для размещения объекта аренды арендодатель формирует карточку объявления, включающую наименование, категорию, техническое описание, фотографические материалы, ценовые условия и периоды доступности. Серверная часть обеспечивает приём и хранение медиафайлов, индексирование текстовых атрибутов для реализации полнотекстового поиска, а также фильтрацию результатов по заданным критериям: категории, ценовому диапазону и датам доступности.

1.3.3 Бронирование и совершение сделки

Процесс оформления аренды автоматизируется на основе встроенного серверного механизма управления бронированиями, исключающего возможность одновременного резервирования одного объекта несколькими пользователями. После выбора арендатором периода аренды серверная часть автоматически вычисляет итоговую стоимость с учётом комиссионного вознаграждения платформы. Проведение расчётов осуществляется исключительно через платформу: денежные средства временно блокируются на стороне сервера до получения подтверждений от обеих сторон о выполнении своих обязательств. Данный механизм гарантирует арендодателю получение вознаграждения, а арендатору — передачу предмета аренды в состоянии, соответствующем условиям договора.

1.3.4 Система рейтингов и отзывов

Механизм репутационной оценки участников является одним из ключевых инструментов снижения информационной асимметрии в С2С-сервисах. По завершении каждой сделки стороны имеют возможность выставить оценку и оставить текстовый отзыв. Серверная часть сохраняет поступившие данные, пересчитывает агрегированный рейтинг пользователя и учитывает его в алгоритме ранжирования объявлений в результатах поиска. Накопленная репутация оказывает непосредственное влияние на видимость предложений и готовность других участников платформы вступать в сделки с данным пользователем.

1.4 Компоненты серверной части программно-информационной системы

Серверная часть онлайн-площадки представляет собой совокупность специализированных программных модулей, реализованных по принципу разделения ответственности и предоставляющих свои функции клиентскому приложению через унифицированный REST API. Взаимодействие модулей осуществляется посредством внутренних интерфейсов; персистентное хранение данных обеспечивается реляционной СУБД.

1.4.1 Модуль управления пользователями

Модуль реализует функции регистрации учётных записей, аутентификации на основе JSON Web Token (JWT) с поддержкой механизма обновления токенов по истечении срока их действия, а также авторизации на основе атрибутивной ролевой модели доступа. В базе данных персистируются профили пользователей, история транзакций, рейтинговые показатели, текстовые отзывы и документы, полученные в ходе верификации личности.

1.4.2 Модуль каталога и поиска

Модуль экспонирует набор API-endpoints для выполнения операций CRUD над записями объявлений. Серверная логика включает загрузку, валидацию и хранение медиафайлов посредством выделенного файлового хранилища, построение и обновление поискового индекса, а также обработку поисковых запросов с применением комбинированной фильтрации по категориям, ценовому диапазону и временным периодам доступности.

1.4.3 Модуль бронирования и управления доступностью

Функциональная ответственность модуля состоит в управлении временной доступностью объектов аренды и обеспечении атомарности операций резервирования с целью исключения конкурентных конфликтов. Серверная часть поддерживает актуальный календарь занятости в разрезе каждого объявления, отслеживает конечный автомат состояний заявки на бронирование (создана, подтверждена, оплачена, завершена, отменена) и инициирует отправку уведомлений при каждом переходе между состояниями.

1.4.4 Модуль аналитики и отчётности

Модуль агрегирует данные о завершённых транзакциях и предоставляет арендодателям персональный дашборд с показателями доходов и статистикой просмотров объявлений. Для арендаторов формируется история бронирований с детализацией по статусам. На уровне администрирования платформы реализовано формирование сводных отчётов о совокупном обороте и количестве совершённых сделок за произвольно заданный временной период.

1.5 Правовое регулирование онлайн-платформ аренды в Российской Федерации

Проектирование серверной части онлайн-площадки осуществляется с учётом действующей нормативно-правовой базы, определяющей правовой

статус участников арендных отношений, требования к обработке персональных данных и порядку проведения финансовых операций.

Гражданский кодекс Российской Федерации, глава 34 «Аренда», закрепляет существенные условия договора аренды, права и обязанности арендодателя и арендатора, а также ответственность за недостатки переданного в пользование имущества[?]. Совокупность данных норм определяет обязательный состав реквизитов и структуру разделов документов, автоматически генерируемых серверным модулем.

Закон РФ от 07.02.1992 № 2300-1 «О защите прав потребителей» распространяется на правоотношения с арендаторами, использующими предмет аренды в личных или бытовых целях. В соответствии с данным законом арендатор наделён правом на получение исчерпывающей и достоверной информации об услуге, на расторжение договора в любой момент с возмещением арендодателю фактически понесённых расходов, а также на компенсацию убытков, возникших вследствие недостатков оказанной услуги. Серверная часть должна обеспечивать неизменяемое хранение условий сделки в том виде, в котором они были приняты сторонами[?].

Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных» обязывает платформу как оператора персональных данных обеспечить законность их сбора и обработки, а также надёжную техническую защиту от несанкционированного доступа. Персональные данные пользователей — паспортные сведения, контактная информация, платёжные реквизиты — подлежат хранению в зашифрованном виде с разграничением доступа на уровне серверной части[?]. Интеграция с платёжными сервисами, прошедшими сертификацию по стандартам PCI DSS или ISO-20022, обеспечивает соответствие требованиям к безопасности финансовых транзакций.

Федеральный закон от 27.11.2018 № 422-ФЗ устанавливает специальный налоговый режим «Налог на профессиональный доход», применение которого доступно арендодателям — физическим лицам, не имеющим статуса индивидуального предпринимателя, в качестве правового основания для легального получения дохода от сдачи имущества в аренду[?].

1.6 Анализ существующих решений и аналогов

С целью выработки обоснованных технических требований к проектируемому серверному решению проведён сравнительный анализ трёх наиболее востребованных отечественных платформ в сегменте онлайн-аренды: Rentomania, Авито и Арендую.ру. Оценка производилась по критериям, непосредственно связанным с функциональностью серверной части. Результаты сопоставления представлены в таблице ??.

Таблица 1.1 – Сравнение существующих решений и аналогов

Параметр	Rentomania	Авито	Арендую.ру	Проектируемая онлайн площадка
Размещение объявлений арендодателем	+	+	+	+
Поиск и фильтрация товаров	+	+	+	+
Онлайн-бронирование	+	—	—	+
Встроенная онлайн-оплата	+	+	—	+
Система рейтинга и отзывов	+	+	—	+

Продолжение таблицы ??

Параметр	Rentomania	Авито	Арендую.ру	Проектируемая онлайн площадка
Чат между арендатором и арендодателем	+	+	—	+
Адаптивная вёрстка для мобильных устройств	+	+	—	+
Верификация пользователей	+	—	—	+
Панель аналитики для арендодателя	—	—	—	+
Понятный интерфейс	—	+	—	+
Уведомления о статусе аренды	+	—	—	+

2 Техническое задание

2.1 Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу бакалавра «Онлайн-площадка для сдачи вещей в аренду. Разработка серверной части».

2.2 Цель и назначение разработки

Целью выпускной квалификационной работы является проектирование, разработка и тестирование серверной части веб-приложения, обеспечивающей хранение данных, бизнес-логику и предоставление REST API для онлайн-площадки аренды вещей. Серверная часть предоставляет арендодателям возможность публиковать объявления, а арендаторам — находить, бронировать и оплачивать аренду с гарантиями безопасности сделки. Задачами данной разработки являются:

- изучение и анализ предметной области;
- проектирование архитектуры серверной части и базы данных;
- разработка REST API веб-приложения;
- интеграция с сервисом DaData;
- тестирование разработанного API.

2.3 Требования к серверной части веб-приложения

2.3.1 Функциональные требования к серверной части

На основании описанных бизнес-процессов и анализа предметной области сформирован перечень функциональных требований к API и серверной части, представленный в таблице ??.

Таблица 2.1 – Функциональные требования к серверной части

№	Функциональное требование	Описание
1	Регистрация и аутентификация	Создание учётной записи с подтверждением email через шести-значный код; выдача пары JWT-токенов при входе; восстановление пароля по коду из письма.
2	Верификация пользователей	Серверная валидация реквизитов в зависимости от типа пользователя: паспорт и ИНН для физических лиц, ИНН и ОГРНИП для ИП, ИНН и КПП для юридических лиц; автозаполнение реквизитов по ИНН через DaData.
3	Управление профилем	API-endpoints чтения и обновления персональных данных; пересчёт признака заполненности профиля; смена пароля.
4	Размещение объявления	Создание записи объявления с атрибутами: наименование, категория, описание, цена за день, залог; назначение владельца; отправка на модерацию.
5	Управление объявлениями	Endpoints редактирования и удаления собственных объявлений с проверкой прав владельца; загрузка и удаление изображений с сохранением в файловом хранилище.

Продолжение таблицы ??

№	Функциональное требование	Описание
6	Фильтрация и поиск	Серверная фильтрация объявлений по категории и периоду доступности с исключением занятых дат из активных бронирований; полнотекстовый поиск по наименованию и описанию.
7	Карточка объявления	Предоставление полных данных объявления: характеристики, изображения, рейтинг, отзывы, занятые периоды аренды, вычисленные из активных бронирований.
8	Онлайн-бронирование	Создание бронирования с проверкой доступности дат; автоматический расчёт стоимости на сервере с учётом наценки платформы 20% и сервисного сбора 5%; управление статусами бронирования.
9	Онлайн-оплата	Формирование демонстрационной СБП-сессии с реквизитами и QR-кодом; обновление статуса оплаты после подтверждения; обработка запроса на возврат средств через endpoint.

Продолжение таблицы ??

№	Функциональное требование	Описание
10	Система отзывов	Создание отзыва по завершённомu или оплаченному бронированию; автоматический пересчёт среднего рейтинга объявления и пользователя после каждого нового отзыва.
11	Аналитика арендодателя	Вычисление статистики на сервере: количество бронирований, чистая прибыль без учёта комиссий и залога, история сделок за период.
12	Административная панель	Модерация объявлений и пользователей, управление категориями, просмотр транзакций и заявок на модерацию через встроенную панель администратора Django.

2.3.2 Требования к программному интерфейсу

Серверная часть должна предоставлять следующие группы API-endpoints:

- модуль пользователей: регистрация, подтверждение email, аутентификация, управление профилем, смена и восстановление пароля, автозаполнение реквизитов по ИНН;

- модуль объявлений: создание, редактирование, удаление, поиск, фильтрация, загрузка и удаление изображений, управление статусами модерации;

– модуль бронирований: создание заявки, подтверждение и отмена, проведение оплаты, подтверждение передачи и возврата товара с фотофиксацией, запрос и обработка возврата средств.

2.4 Моделирование вариантов использования

Для серверной части разработана модель вариантов использования на языке UML, отражающая взаимодействие пользователей с программным интерфейсом системы. На основании анализа предметной области реализованы следующие прецеденты:

1. Регистрация с подтверждением email.
2. Аутентификация с JWT-токен.
3. Восстановление пароля.
4. Просмотр профиля.
5. Обновление профиля.
6. Автозаполнение реквизитов по ИНН.
7. Просмотр каталога объявлений.
8. Фильтрация и поиск объявлений.
9. Просмотр карточки объявления.
10. Создание объявления.
11. Редактирование объявления.
12. Удаление объявления.
13. Загрузка изображений.
14. Удаление изображений.
15. Модерация объявлений.
16. Создание категории.
17. Удаление категории.
18. Блокировка пользователя.
19. Разблокировка пользователя.
20. Блокировка объявления.
21. Разблокировка объявления.
22. Создание бронирования.

23. Просмотр бронирований.
24. Подтверждение бронирования.
25. Отмена бронирования.
26. Завершение аренды.
27. Оплата через СБП.
28. Подтверждение получения товара.
29. Подтверждение возврата товара.
30. Возврат залога.
31. Возврат средств через СБП.
32. Создание отзыва.

Разрабатываемое веб-приложение имеет 3 группы пользователей с разными правами: гость, пользователь, администратор.

Гостю должны быть доступны следующие функции:

1. Регистрация с подтверждением email.
2. Аутентификация с JWT-токен.
3. Восстановление пароля.
4. Автозаполнение реквизитов по ИНН.
5. Просмотр каталога объявлений.
6. Фильтрация и поиск объявлений.
7. Просмотр карточки объявления.

На рисунке ?? изображена диаграмма прецедентов для гостя.

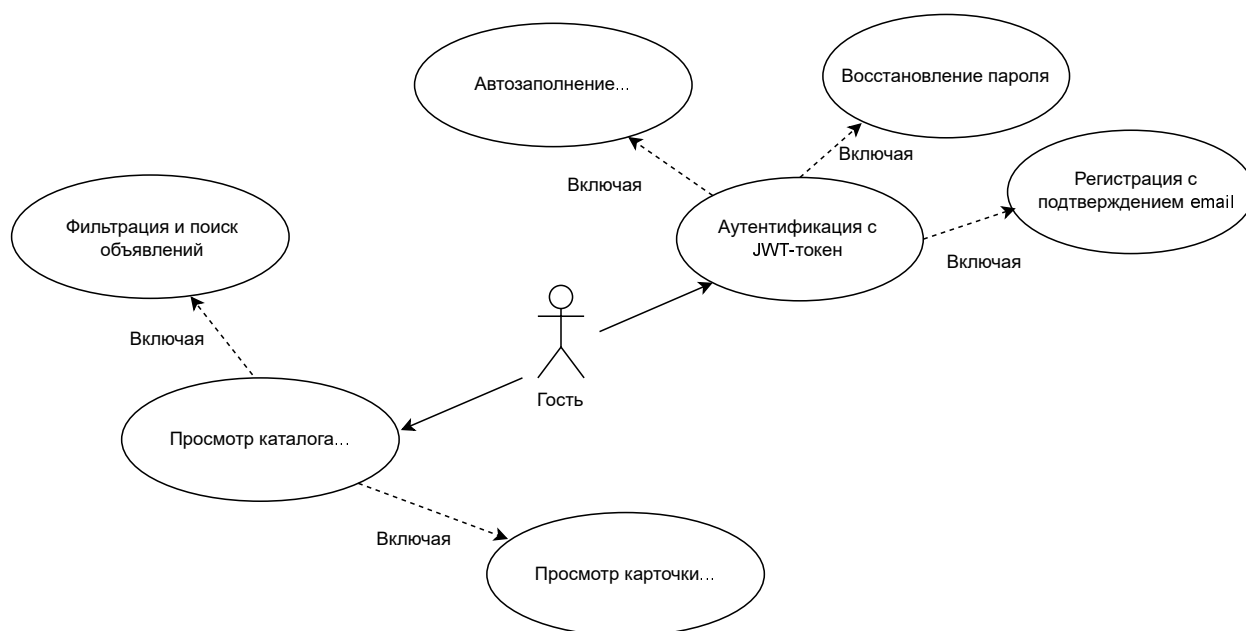


Рисунок 2.1 – Диаграмма прецедентов для гостя

Пользователю должны быть доступны следующие функции:

1. Просмотр профиля.
2. Обновление профиля.
3. Автозаполнение реквизитов по ИНН.
4. Просмотр каталога объявлений.
5. Фильтрация и поиск объявлений.
6. Просмотр карточки объявления.
7. Создание объявления.
8. Редактирование объявления.
9. Удаление объявления.
10. Загрузка изображений.
11. Удаление изображений.
12. Создание бронирования.
13. Просмотр бронирований.
14. Подтверждение бронирования.
15. Отмена бронирования.
16. Завершение аренды.
17. Оплата через СБП.
18. Подтверждение получения товара.

19. Подтверждение возврата товара.
20. Возврат залога.
21. Возврат средств через СБП.
22. Создание отзыва.

На рисунке ?? изображена диаграмма прецедентов для пользователя.

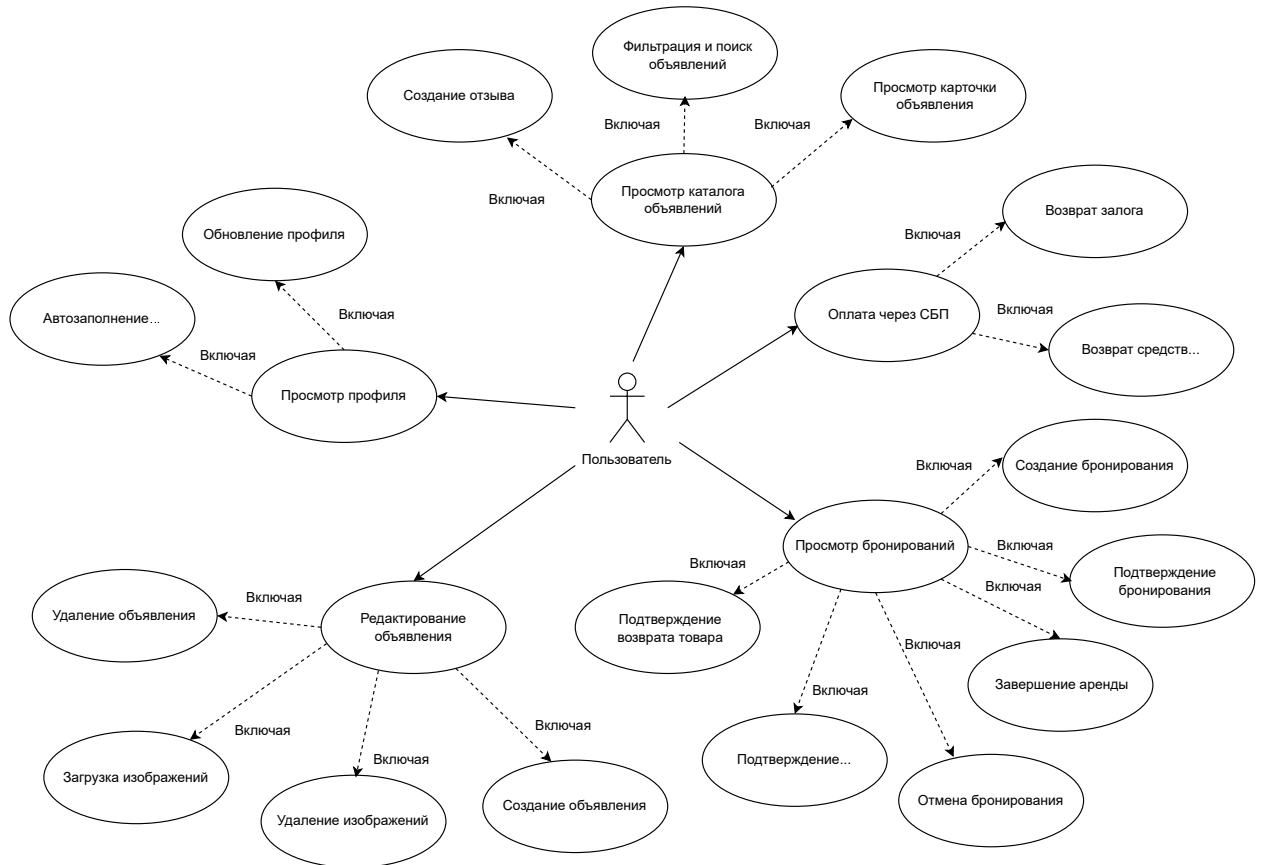


Рисунок 2.2 – Диаграмма прецедентов для пользователя

Администратору должны быть доступны следующие функции:

1. Просмотр профиля.
2. Просмотр каталога объявлений.
3. Фильтрация и поиск объявлений.
4. Просмотр карточки объявления.
5. Просмотр бронирований.
6. Модерация объявлений.
7. Создание категории.
8. Удаление категории.

9. Блокировка пользователя.
10. Разблокировка пользователя.
11. Блокировка объявления.
12. Разблокировка объявления.

На рисунке ?? изображена диаграмма прецедентов для администратора.

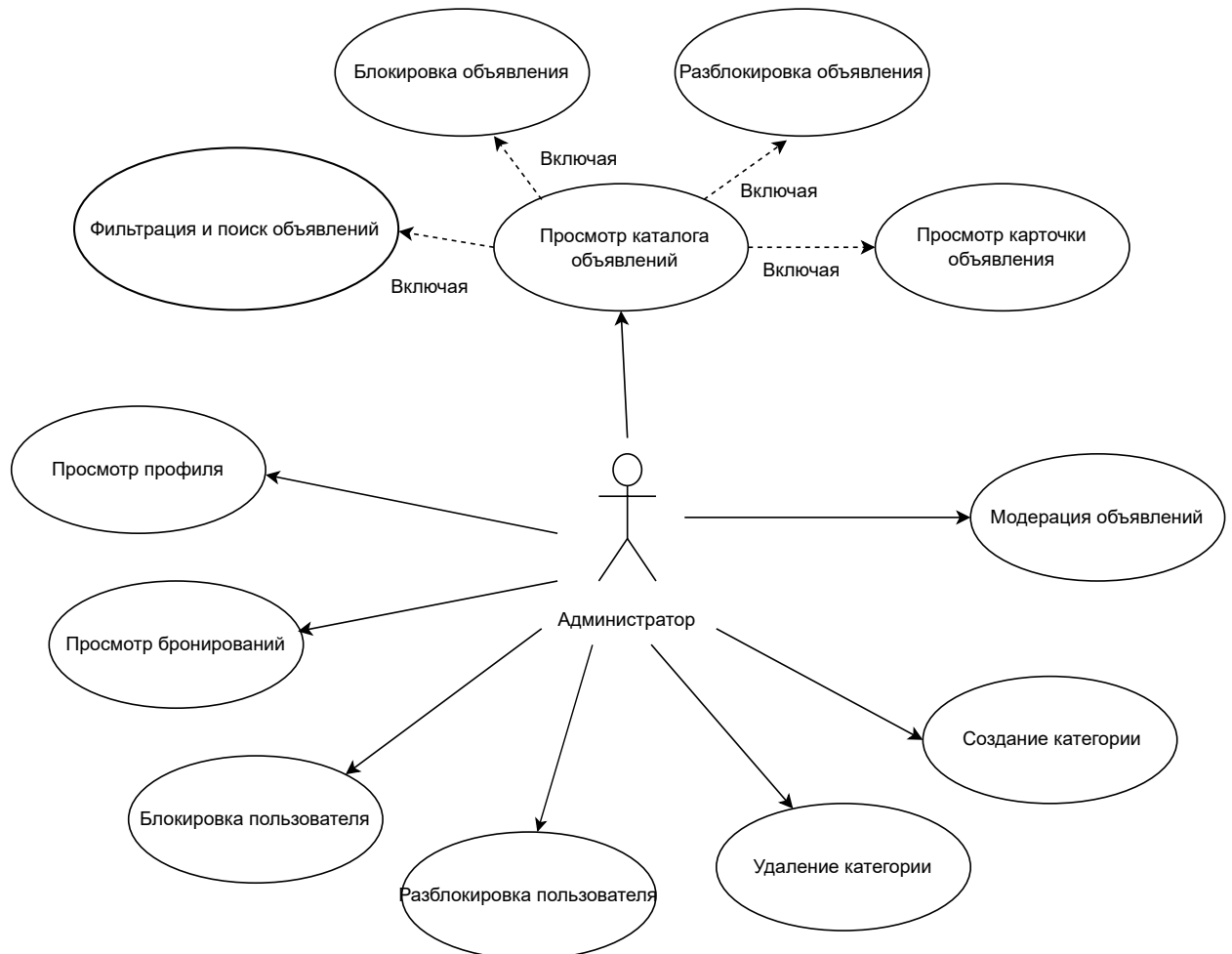


Рисунок 2.3 – Диаграмма прецедентов для администратора

2.4.1 Регистрация с подтверждением email

Заинтересованные лица и их требования: пользователь желает создать учётную запись для получения доступа к функциям сервиса аренды вещей. Предусловие: пользователь обращается к API с корректными данными; указанный email не зарегистрирован в системе. Постусловие: в базе данных создана новая учётная запись; email подтверждён. Основной успешный сценарий:

1. Клиент отправляет POST-запрос с адресом электронной почты.
2. Сервер проверяет уникальность email в базе данных.
3. Сервер генерирует шестизначный код подтверждения и сохраняет его в кэше с таймаутом 15 минут.
4. Сервер отправляет код на указанный email через SMTP.
5. Клиент отправляет POST-запрос с именем пользователя, email, паролем, типом аккаунта, реквизитами и кодом подтверждения из письма.
6. Сервер проверяет соответствие кода сохранённому значению и его срок действия.
7. Сервер валидирует реквизиты в соответствии с типом пользователя: для физического лица — ФИО и паспортные данные; для ИП — наименование, ИНН и ОГРНИП; для юридического лица — наименование, ИНН и КПП.
8. Сервер создаёт учётную запись, сохраняет её в базе данных и удаляет использованный код из кэша.
9. Сервер возвращает клиенту данные созданного пользователя.

2.4.2 Аутентификация и JWT-токен

Заинтересованные лица и их требования: пользователь желает войти в систему для получения доступа к защищённым ресурсам API. Предусловие: учётная запись пользователя существует в базе данных. Постусловие: пользователь получает пару JWT-токенов для последующей авторизации запросов. Основной успешный сценарий:

1. Пользователь направляет серверу логин и пароль.
2. Сервер проверяет соответствие переданных учётных данных записи в базе данных.
3. Сервер формирует и возвращает access-токен с ограниченным сроком действия и refresh-токен для его обновления.
4. При последующих обращениях к защищённым endpoints клиент передаёт access-токен в заголовке запроса.

5. При истечении срока действия access-токена клиент использует refresh-токен для получения нового.

2.4.3 Восстановление пароля

Заинтересованные лица и их требования: пользователь забыл пароль и желает восстановить доступ к учётной записи. Предусловие: учётная запись с указанным email существует в базе данных. Постусловие: пароль пользователя изменён на новый; использованный код удалён из кэша. Основной успешный сценарий:

1. Пользователь направляет серверу запрос с адресом электронной почты.
2. Сервер проверяет наличие учётной записи с указанным email.
3. Сервер генерирует шестизначный код и сохраняет его в кэше с таймаутом 15 минут.
4. Сервер отправляет код на email пользователя.
5. Пользователь направляет серверу запрос с email, кодом из письма и новым паролем.
6. Сервер проверяет соответствие кода сохранённому значению и его срок действия.
7. Сервер устанавливает новый пароль и удаляет использованный код из кэша.

2.4.4 Просмотр профиля

Заинтересованные лица и их требования: авторизованный пользователь желает получить актуальные данные своей учётной записи. Предусловие: пользователь авторизован в системе. Постусловие: сервер возвращает данные профиля текущего пользователя. Основной успешный сценарий:

1. Пользователь запрашивает данные своего профиля.
2. Сервер извлекает идентификатор пользователя из токена и находит его запись в базе данных.

3. Сервер сериализует данные профиля: имя пользователя, адрес электронной почты, тип аккаунта, реквизиты и признак заполненности профиля.
4. Сервер возвращает сериализованные данные в формате JSON.

2.4.5 Обновление профиля

Заинтересованные лица и их требования: пользователь желает обновить данные своей учётной записи. Предусловие: пользователь авторизован. Постусловие: данные профиля обновлены в базе данных; признак заполненности профиля пересчитан. Основной успешный сценарий:

1. Пользователь направляет серверу обновляемые поля профиля.
2. Сервер валидирует переданные поля в соответствии с типом пользователя.
3. Сервер сохраняет изменения и пересчитывает признак заполненности профиля.
4. Сервер возвращает обновлённые данные профиля.

2.4.6 Автозаполнение реквизитов по ИНН

Заинтересованные лица и их требования: ИП и юридические лица хотят автоматически заполнить реквизиты по ИНН без ручного ввода. Предусловие: пользователь знает свой ИНН; внешний сервис DaData доступен. Постусловие: сервер возвращает реквизиты, соответствующие переданному ИНН. Основной успешный сценарий:

1. Пользователь направляет серверу запрос с ИНН.
2. Сервер обращается к внешнему сервису DaData и определяет тип субъекта по длине ИНН: 10 цифр — юридическое лицо, 12 цифр — ИП.
3. Сервер возвращает наименование организации и соответствующие реквизиты: для юридического лица — ИНН и КПП, для ИП — ИНН и ОГРНИП.
4. Пользователь использует полученные данные для заполнения профиля.

2.4.7 Просмотр каталога объявлений

Заинтересованные лица и их требования: пользователь желает получить список вещей, доступных для аренды. Предусловие: запрос может выполнять как авторизованный, так и неавторизованный пользователь. Постусловие: сервер возвращает список опубликованных объявлений. Основной успешный сценарий:

1. Пользователь запрашивает список объявлений.
2. Сервер формирует запрос к базе данных: для неавторизованного пользователя выбирает только объявления со статусом «Опубликовано», для авторизованного — также включает его собственные объявления в любом статусе.
3. Сервер сериализует данные: наименование, описание, цену за день, залог, категорию, средний рейтинг, количество отзывов и ссылки на изображения.
4. Сервер возвращает постраничный список объявлений в формате JSON.

2.4.8 Фильтрация и поиск объявлений

Заинтересованные лица и их требования: пользователь желает получить только те объявления, которые соответствуют заданным критериям. Предусловие: пользователь передаёт параметры фильтрации или поиска в запросе. Постусловие: сервер возвращает отфильтрованный список объявлений. Основной успешный сценарий:

1. Пользователь передаёт параметры в запросе: категорию, текст поиска, порядок сортировки или минимальный рейтинг.
2. Сервер применяет фильтр по идентификатору категории и полнотекстовый поиск по названию и описанию.
3. Сервер применяет указанную сортировку по цене, дате создания или рейтингу.

4. Сервер возвращает отфильтрованный постраничный список объявлений.

2.4.9 Просмотр карточки объявления

Заинтересованные лица и их требования: пользователь желает получить полные данные о вещи перед оформлением аренды. Предусловие: объявление существует в базе данных. Постусловие: сервер возвращает полный набор данных по объявлению, включая рейтинг, отзывы и занятые периоды. Основной успешный сценарий:

1. Пользователь запрашивает данные конкретного объявления по его идентификатору.
2. Сервер выбирает запись объявления вместе с данными владельца, категории и изображений.
3. Сервер прикрепляет список отзывов и средний рейтинг объявления и его владельца.
4. Сервер формирует список занятых периодов на основе активных бронирований со статусами «Рассматривается» и «Подтверждено».
5. Сервер возвращает полный набор данных в формате JSON.

2.4.10 Создание объявления

Заинтересованные лица и их требования: авторизованный пользователь с заполненным профилем желает разместить вещь для аренды. Предусловие: пользователь авторизован; признак заполненности профиля имеет значение «заполнен». Постусловие: объявление создано в базе данных со статусом «На модерации»; заявка на модерацию зафиксирована. Основной успешный сценарий:

1. Пользователь направляет серверу POST-запрос с данными объявления: наименование, описание, цена за день, залог, категория.
2. Сервер проверяет заполненность профиля пользователя; при незаполненном профиле возвращает ошибку.

3. Сервер создаёт запись объявления со статусом «На модерации» и назначает пользователя владельцем.

4. Сервер создаёт заявку на модерацию с типом действия «Создание».

5. Сервер возвращает созданное объявление с присвоенным идентификатором.

2.4.11 Редактирование объявления

Заинтересованные лица и их требования: владелец желает изменить данные ранее созданного объявления. Предусловие: пользователь авторизован, является владельцем объявления и имеет заполненный профиль. Постусловие: данные объявления обновлены в базе данных; создана заявка на повторную модерацию. Основной успешный сценарий:

1. Пользователь направляет серверу запрос с изменёнными полями объявления.

2. Сервер проверяет, что текущий пользователь является владельцем данного объявления; в противном случае возвращает ошибку доступа.

3. Сервер сохраняет обновлённые данные, переводит статус объявления в «На модерации» и создаёт заявку на модерацию с типом действия «Редактирование».

4. Сервер возвращает обновлённую карточку объявления.

2.4.12 Удаление объявления

Заинтересованные лица и их требования: владелец желает удалить объявление из системы. Предусловие: пользователь авторизован и является владельцем объявления. Постусловие: объявление и связанные изображения удалены из базы данных и файлового хранилища. Основной успешный сценарий:

1. Пользователь направляет серверу DELETE-запрос на удаление объявления.

2. Сервер проверяет, что текущий пользователь является владельцем данного объявления; в противном случае возвращает ошибку доступа.

3. Сервер удаляет объявление и связанные с ним изображения из базы данных и файлового хранилища.
4. Сервер возвращает подтверждение успешного удаления.

2.4.13 Загрузка изображений

Заинтересованные лица и их требования: владелец желает прикрепить фотографии вещи к объявлению. Предусловие: пользователь авторизован и является владельцем объявления. Постусловие: изображение сохранено в файловом хранилище и связано с объявлением в базе данных. Основной успешный сценарий:

1. Пользователь направляет серверу POST-запрос с файлом изображения и идентификатором объявления в формате multipart/form-data.
2. Сервер проверяет, что текущий пользователь является владельцем указанного объявления; в противном случае возвращает ошибку доступа.
3. Сервер сохраняет файл в файловом хранилище.
4. Сервер создаёт запись, связывающую изображение с объявлением.
5. Сервер возвращает идентификатор и ссылку на загруженное изображение.

2.4.14 Удаление изображений

Заинтересованные лица и их требования: владелец желает удалить изображение из своего объявления. Предусловие: пользователь авторизован и является владельцем объявления; изображение привязано к данному объявлению. Постусловие: изображение удалено из базы данных и файлового хранилища. Основной успешный сценарий:

1. Пользователь направляет серверу DELETE-запрос на удаление конкретного изображения.
2. Сервер проверяет, что изображение принадлежит объявлению текущего пользователя; в противном случае возвращает ошибку доступа.
3. Сервер удаляет файл из хранилища и запись из базы данных.
4. Сервер возвращает подтверждение успешного удаления.

2.4.15 Модерация объявлений

Заинтересованные лица и их требования: администратор желает проверить поступившие заявки на публикацию объявлений и принять по ним решение. Предусловие: в системе существуют заявки на модерацию со статусом «Ожидает рассмотрения»; пользователь обладает правами суперпользователя. Постусловие: статус заявки и связанного объявления обновлены; владелец уведомлён о результате. Основной успешный сценарий:

1. Администратор запрашивает список заявок на модерацию, при необходимости фильтруя по статусу.
2. Сервер возвращает заявки с полными данными объявления, включая изображения, описание и реквизиты владельца.
3. Администратор принимает решение об одобрении: сервер переводит статус объявления в «Опубликовано» и статус заявки в «Одобрено».
4. При отклонении администратор направляет запрос с указанием причины: сервер переводит статус объявления в «Отклонено», статус заявки в «Отклонено» и сохраняет причину отказа.

2.4.16 Создание категории

Заинтересованные лица и их требования: администратор желает добавить новую категорию товаров для структурирования каталога. Предусловие: пользователь авторизован и обладает правами суперпользователя. Постусловие: категория создана в базе данных и доступна для выбора при размещении объявлений. Основной успешный сценарий:

1. Администратор направляет серверу POST-запрос с наименованием новой категории.
2. Сервер проверяет права доступа пользователя.
3. Сервер создаёт запись категории в базе данных.
4. Сервер возвращает созданную категорию с присвоенным идентификатором.

2.4.17 Удаление категории

Заинтересованные лица и их требования: администратор желает удалить устаревшую или ошибочно созданную категорию. Предусловие: пользователь авторизован и обладает правами суперпользователя; категория существует в базе данных. Постусловие: категория удалена из базы данных; связанные объявления остаются без категории. Основной успешный сценарий:

1. Администратор направляет серверу DELETE-запрос на удаление категории по её идентификатору.
2. Сервер проверяет права доступа пользователя.
3. Сервер удаляет запись категории из базы данных.
4. Сервер возвращает подтверждение успешного удаления.

2.4.18 Блокировка пользователя

Заинтересованные лица и их требования: администратор желает ограничить доступ недобросовестного пользователя к платформе. Предусловие: пользователь обладает правами суперпользователя; целевая учётная запись существует в базе данных и не является суперпользователем. Постусловие: признак активности учётной записи установлен в значение «неактивен»; пользователь лишается возможности аутентификации. Основной успешный сценарий:

1. Администратор направляет серверу запрос на блокировку пользователя с указанием его идентификатора.
2. Сервер проверяет права доступа и убеждается, что целевой пользователь не является суперпользователем.
3. Сервер устанавливает признак активности учётной записи в значение «неактивен».
4. Сервер возвращает подтверждение выполненной блокировки.

2.4.19 Разблокировка пользователя

Заинтересованные лица и их требования: администратор желает восстановить доступ ранее заблокированного пользователя к платформе. Предусловие: пользователь обладает правами суперпользователя; целевая учётная запись существует в базе данных и имеет статус «неактивен». Постусловие: признак активности учётной записи установлен в значение «активен»; пользователь восстанавливает возможность аутентификации. Основной успешный сценарий:

1. Администратор направляет серверу запрос на разблокировку пользователя с указанием его идентификатора.
2. Сервер проверяет права доступа.
3. Сервер устанавливает признак активности учётной записи в значение «активен».
4. Сервер возвращает подтверждение восстановления доступа.

2.4.20 Блокировка объявления

Заинтересованные лица и их требования: администратор желает скрыть объявление, нарушающее правила платформы, из публичного каталога. Предусловие: пользователь обладает правами суперпользователя; объявление существует в базе данных. Постусловие: статус объявления переведён в «Заблокировано»; объявление не отображается в каталоге для других пользователей. Основной успешный сценарий:

1. Администратор направляет серверу запрос на блокировку объявления с указанием его идентификатора.
2. Сервер проверяет права доступа пользователя.
3. Сервер переводит статус объявления в «Заблокировано».
4. Сервер возвращает подтверждение выполненной блокировки.

2.4.21 Разблокировка объявления

Заинтересованные лица и их требования: администратор желает восстановить видимость ранее заблокированного объявления в публичном каталоге. Предусловие: пользователь обладает правами суперпользователя; объявление существует в базе данных и имеет статус «Заблокировано». Постусловие: статус объявления переведён в «Опубликовано»; объявление снова отображается в каталоге. Основной успешный сценарий:

1. Администратор направляет серверу запрос на разблокировку объявления с указанием его идентификатора.
2. Сервер проверяет права доступа пользователя.
3. Сервер переводит статус объявления в «Опубликовано».
4. Сервер возвращает подтверждение восстановления видимости объявления.

2.4.22 Создание бронирования

Заинтересованные лица и их требования: авторизованный пользователь желает забронировать чужую вещь; итоговая стоимость должна рассчитываться автоматически на стороне сервера. Предусловие: пользователь авторизован, его профиль заполнен; пользователь не является владельцем выбранного объявления. Постусловие: бронирование создано; итоговая стоимость с разбивкой по компонентам сохранена в базе данных. Основной успешный сценарий:

1. Пользователь направляет серверу POST-запрос с идентификатором объявления и датами начала и окончания аренды.
2. Сервер проверяет, что пользователь не является владельцем вещи и что его профиль заполнен.
3. Сервер проверяет отсутствие пересечения запрошенного периода с существующими активными бронированиями.
4. Сервер рассчитывает количество дней аренды и базовую стоимость.

5. Сервер применяет наценку платформы 20% и сервисный сбор 5%, фиксирует сумму залога из объявления.

6. Сервер сохраняет бронирование со статусом «Рассматривается» и отдельными полями: сумма аренды, сервисный сбор, сумма залога и итоговая сумма.

7. Сервер возвращает созданное бронирование с полной разбивкой стоимости.

2.4.23 Просмотр бронирований

Заинтересованные лица и их требования: пользователь желает просматривать как заявки на аренду своих вещей, так и заявки, оформленные им самим. Предусловие: пользователь авторизован. Постусловие: сервер возвращает список бронирований с возможностью фильтрации по роли. Основной успешный сценарий:

1. Пользователь запрашивает список бронирований, при необходимости указывая роль: входящие или исходящие.

2. Сервер выбирает из базы данных бронирования, в которых текущий пользователь является арендатором или владельцем вещи.

3. При наличии параметра роли сервер дополнительно фильтрует список: входящие — только заявки на собственные вещи, исходящие — только заявки пользователя.

4. Сервер сериализует данные: вещь, даты, стоимость с разбивкой по компонентам, статус бронирования, статус оплаты, статус залога.

5. Сервер возвращает постраничный список бронирований в формате JSON.

2.4.24 Подтверждение бронирования

Заинтересованные лица и их требования: арендодатель желает принять входящую заявку на аренду своей вещи. Предусловие: бронирование имеет статус «Рассматривается»; текущий пользователь является владельцем вещи.

Постусловие: статус бронирования изменён на «Подтверждено»; арендатор уведомлён. Основной успешный сценарий:

1. Арендодатель направляет серверу POST-запрос на подтверждение бронирования.
2. Сервер проверяет, что текущий пользователь является владельцем данной вещи; в противном случае возвращает ошибку доступа.
3. Сервер переводит статус бронирования в «Подтверждено».
4. Сервер возвращает обновлённые данные бронирования.

2.4.25 Отмена бронирования

Заинтересованные лица и их требования: арендатор или арендодатель желают отменить бронирование; арендатор может указать причину отказа. Предусловие: бронирование существует в базе данных и находится в допустимом для отмены статусе. Постусловие: статус бронирования изменён на «Отменено»; причина отказа сохранена при наличии. Основной успешный сценарий:

1. Участник сделки направляет серверу POST-запрос на отмену бронирования.
2. Сервер проверяет, что текущий пользователь является арендатором или владельцем данного бронирования; в противном случае возвращает ошибку доступа.
3. Сервер переводит статус бронирования в «Отменено».
4. При отказе арендатора с указанием причины сервер дополнительно сохраняет текст причины отказа.
5. Сервер возвращает обновлённые данные бронирования.

2.4.26 Завершение аренды

Заинтересованные лица и их требования: владелец желает завершить аренду после возврата вещи арендатором. Предусловие: бронирование имеет статус «Подтверждено» и статус оплаты «Оплачено». Постусловие: статус бронирования изменён на «Завершено». Основной успешный сценарий:

1. Арендодатель направляет серверу POST-запрос на завершение аренды.
2. Сервер проверяет, что текущий пользователь является владельцем данной вещи.
3. Сервер проверяет, что бронирование оплачено.
4. Сервер переводит статус бронирования в «Завершено».
5. Сервер возвращает обновлённые данные бронирования.

2.4.27 Оплата через СБП

Заинтересованные лица и их требования: арендатор желает оплатить подтверждённое бронирование через демонстрационный механизм СБП. Предусловие: бронирование имеет статус «Подтверждено»; оплата ещё не произведена. Постусловие: статус оплаты бронирования изменён на «Оплачено»; при наличии залога его статус также изменён на «Оплачен». Основной успешный сценарий:

1. Арендатор направляет POST-запрос на запуск оплаты.
2. Сервер проверяет статус бронирования, его принадлежность данному арендатору и отсутствие действующей платёжной сессии.
3. Сервер формирует демонстрационную СБП-сессию: генерирует идентификатор платежа, устанавливает срок действия 15 минут и формирует реквизиты платежа (сумма, получатель, банк, QR-данные).
4. Сервер возвращает параметры платежа арендатору.
5. Арендатор подтверждает факт оплаты, направляя POST-запрос.
6. Сервер проверяет актуальность платёжной сессии; при истечении срока переводит статус в «Ошибка».
7. Сервер устанавливает статус оплаты «Оплачено», фиксирует дату оплаты и, при наличии залога, переводит его статус в «Оплачен».

2.4.28 Подтверждение получения товара

Заинтересованные лица и их требования: арендатор желает зафиксировать факт получения вещи в начале аренды. Предусловие: бронирование име-

ет статус «Подтверждено» и статус оплаты «Оплачено». Постусловие: факт получения товара зафиксирован в базе данных с фотографией и временной меткой. Основной успешный сценарий:

1. Арендатор направляет POST-запрос с прикрепленной фотографией в формате multipart/form-data.
2. Сервер проверяет, что текущий пользователь является арендатором данного бронирования и что оплата произведена.
3. Сервер сохраняет фотографию в файловом хранилище.
4. Сервер устанавливает признак получения товара арендатором и фиксирует дату и время подтверждения.
5. Сервер возвращает обновлённые данные бронирования.

2.4.29 Подтверждение возврата товара

Заинтересованные лица и их требования: арендодатель желает зафиксировать факт возврата вещи по окончании аренды. Предусловие: арендатор ранее подтвердил получение товара; бронирование имеет статус «Подтверждено». Постусловие: факт возврата товара зафиксирован в базе данных с фотографией и временной меткой; бронирование автоматически переведено в статус «Завершено». Основной успешный сценарий:

1. Арендодатель направляет POST-запрос с фотографией возвращённого товара в формате multipart/form-data.
2. Сервер проверяет, что текущий пользователь является владельцем данной вещи.
3. Сервер проверяет, что арендатор ранее подтвердил получение товара.
4. Сервер сохраняет фотографию в файловом хранилище.
5. Сервер устанавливает признак возврата товара и автоматически переводит бронирование в статус «Завершено».
6. Сервер возвращает обновлённые данные бронирования.

2.4.30 Возврат залога

Заинтересованные лица и их требования: владелец желает вернуть залог арендатору после завершения аренды. Предусловие: бронирование завершено; статус залога имеет значение «Оплачен». Постусловие: статус залога изменён на «Возвращён». Основной успешный сценарий:

1. Арендодатель направляет POST-запрос на возврат залога.
2. Сервер проверяет, что текущий пользователь является владельцем данной вещи.
3. Сервер проверяет, что залог был оплачен.
4. Сервер переводит статус залога в «Возвращён» и возвращает обновлённые данные бронирования.

2.4.31 Возврат средств через СБП

Заинтересованные лица и их требования: арендатор желает вернуть оплаченные средства в случае невозможности исполнения договора аренды; арендодатель принимает решение об одобрении или отклонении запроса. Предусловие: бронирование имеет статус оплаты «Оплачено»; запрос на возврат ещё не подавался. Постусловие: при одобрении статус возврата изменён на «Выполнен»; при наличии залога его статус изменён на «Возвращён». Основной успешный сценарий:

1. Арендатор направляет POST-запрос на возврат средств с указанием причины.
2. Сервер проверяет, что текущий пользователь является арендатором и что запрос на возврат ранее не подавался.
3. Сервер фиксирует запрос: сохраняет причину, дату обращения и переводит статус возврата в значение «Запрошен».
4. Арендодатель получает уведомление и направляет POST-запрос с признаком одобрения.
5. Сервер проверяет, что текущий пользователь является владельцем вещи.

6. При одобрении сервер переводит статус возврата в «Выполнен» и, при наличии оплаченного залога, его статус в «Возвращён».

7. При отклонении сервер переводит статус возврата в «Отклонён»; средства остаются у арендодателя.

2.4.32 Создание отзыва

Заинтересованные лица и их требования: арендатор желает оставить оценку после завершения аренды. Предусловие: бронирование имеет статус «Завершено» либо статус «Подтверждено» при статусе оплаты «Оплачено»; отзыв по данному бронированию ещё не создавался. Постусловие: отзыв сохранён; средний рейтинг объявления и его владельца пересчитан. Основной успешный сценарий:

1. Арендатор направляет серверу POST-запрос с оценкой от 1 до 5 и текстом комментария.

2. Сервер проверяет, что текущий пользователь является арендатором данного бронирования.

3. Сервер проверяет статус бронирования и отсутствие существующего отзыва по нему.

4. Сервер сохраняет отзыв в базе данных.

5. Сервер пересчитывает средний рейтинг объявления и его владельца на основе всех видимых отзывов.

2.5 Требования к оформлению документации

Разработка программной документации и программного изделия должна производиться согласно ГОСТ 19.102-77 и ГОСТ 34.601-90. Единая система программной документации.

3 Технический проект

3.1 Архитектура онлайн-площадки для сдачи вещей в аренду

Необходимо спроектировать и разработать веб-приложение для онлайн площадки для сдачи вещей в аренду. Проектируемая программа представляет собой веб-приложение, построенное по архитектуре «клиент – сервер». Клиентская часть представлена веб-интерфейсом на HTML/CSS/JavaScript, который открывается в браузере пользователя. Серверная часть реализована на языке Python с использованием веб-фреймворка Django и предоставляет REST API для чтения и изменения данных. Хранение структурированных данных выполняется в СУБД PostgreSQL, кэширование и управление WebSocket-соединениями — в Redis. Развёртывание всех компонентов серверной части (Django, PostgreSQL, Redis, Nginx) выполняется с использованием системы контейнеризации Docker, что обеспечивает идентичность окружений на этапах разработки, тестирования и эксплуатации.

Общая схема взаимодействия компонентов приведена на рисунке ??

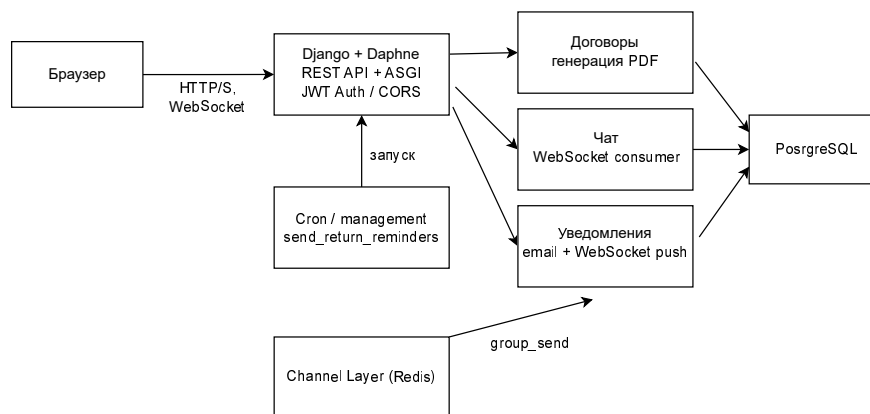


Рисунок 3.1 – Общая схема взаимодействия компонентов

Выбор такой архитектуры обоснован требованиями проекта: необходимо обеспечить одновременную работу пользовательской части (просмотр товаров, бронирование, чаты) и административной части с единой базой данных. Сервер централизует бизнес-логику, валидацию, контроль пересечения дат бронирования, а также статусную модель аренды.

3.2 Обоснование выбора технологии проектирования

При выборе технологического стека учитывались требования простоты развертывания, поддерживаемости, возможности работы с пользовательскими данными и транзакционного хранения сведений о товарах, бронированиях и договорах. Выбранный стек состоит из Python/Django на сервере, HTML/CSS/JavaScript на клиенте, PostgreSQL в качестве СУБД, Docker для контейнеризации, Nginx в качестве reverse-proxy сервера, а также Gunicorn и Daphne для обработки синхронных и асинхронных запросов соответственно.

3.2.1 Язык программирования Python

Python — мультипарадигменный, высокоуровневый язык программирования общего назначения с динамической строгой типизацией и автоматическим управлением памятью. Данный язык программирования широко используется в интернет-приложениях, разработке программного обеспечения, науке о данных и машинном обучении.

3.2.2 Веб-фреймворк Django

Django — это высокоуровневый веб-фреймворк на языке Python, предназначенный для быстрой разработки безопасных и масштабируемых веб-приложений. Он реализует архитектурный подход MVC/MVT и предоставляет разработчику набор встроенных инструментов для работы с маршрутизацией, моделями данных, шаблонами, аутентификацией, административной панелью и защитой от распространенных уязвимостей.

3.2.3 Система управления базами данных PostgreSQL

PostgreSQL — высокопроизводительная объектно-реляционная система управления базами данных с открытым исходным кодом. Она отличается высокой надежностью, поддержкой транзакций, расширенными возможностями работы с данными и соответствием современным требованиям к промышленным СУБД.

3.2.4 Система контейнеризации Docker

Docker — платформа для разработки, доставки и запуска приложений в изолированных контейнерах. Контейнеризация позволяет упаковать приложение со всеми его зависимостями в единый образ, который может быть воспроизведён на любой инфраструктуре. Использование Docker обеспечивает идентичность окружений на этапах разработки, тестирования и эксплуатации, упрощает развёртывание на сервере и масштабирование системы путём запуска нескольких экземпляров контейнеров.

3.2.5 Сервер-обработчик Nginx

Nginx — это высокопроизводительный веб-сервер и сервер-обработчик, широко применяемый при развёртывании веб-приложений. Он используется для приема входящих HTTP-запросов, передачи их внутренним серверным компонентам, а также для обслуживания статических файлов.

3.2.6 WSGI-сервер Gunicorn

Gunicorn — это WSGI-сервер, предназначенный для запуска Python-веб-приложений в производственной среде. Он обеспечивает взаимодействие между веб-сервером и приложением, написанным на Django, и позволяет эффективно обслуживать множество запросов одновременно.

3.2.7 ASGI-сервер Daphne

Daphne — это ASGI-сервер для запуска асинхронных Python-веб-приложений. ASGI (Asynchronous Server Gateway Interface) является надстройкой над WSGI, добавляющей поддержку асинхронных протоколов, включая WebSocket, HTTP/2 и Server-Sent Events.

3.3 Архитектура серверного приложения

Серверная часть веб-приложения реализована как модульное Django-приложение с REST API и поддержкой обмена данными в реальном времени. Архитектура ориентирована на разделение ответственности между модулями: пользователи, каталог товаров, бронирования, договоры, чаты и уведомления. Такой подход обеспечивает управляемость кода и позволяет расширять функциональность без потери основного функционала.

3.3.1 Структурная декомпозиция и порядок запуска

Проект организован в виде набора Django-моделей: `users`, `items`, `bookings`, `contracts`, `chats`, `notifications`. Централизованная конфигурация находится в основном модуле, где задаются настройки, маршрутизация, ASGI/WSGI-точки входа и подключение инфраструктурных компонентов. Подсистема `users` отвечает за регистрацию, профиль и аутентификацию; `items` реализует каталог объявлений и изображения; `bookings` содержит ключевую бизнес-логику аренды, статусов и оплаты; `contracts` формирует договор и сценарий демонстрационной ЭЦП; `chats` и `notifications` обеспечивают коммуникацию и событийную модель.

Точкой входа в серверное приложение является модуль `manage.py`, который инициирует запуск Django-проекта. При старте выполняются следующие этапы:

- чтение переменных окружения;
- применение миграций базы данных;
- сбор статических файлов;
- запуск Gunicorn (для синхронных HTTP-запросов);
- запуск Daphne (для асинхронных WebSocket-соединений);
- запуск reverse-proxy сервера Nginx.

3.3.2 Диаграмма компонентов

На рисунке 3.1 представлена диаграмма компонентов проектируемого веб-приложения. На данной диаграмме изображены:

1. Компонент «Nginx» является внешней точкой входа серверной части. Данный компонент принимает клиентский трафик и передает его во внутренний прикладной слой. Через Nginx проходит как REST-взаимодействие, так и WebSocket-соединения.

2. Компонент «Django» выступает центральным прикладным компонентом, в котором реализованы бизнес-правила, маршрутизация API и обработка real-time событий. Через этот компонент организовано взаимодействие со всеми предметными модулями, а также с инфраструктурными сервисами хранения и доставки данных.

3. Компонент «Модуль пользователя» используется для регистрации, авторизации и работы с профилем пользователя. Он обеспечивает проверку принадлежности операций конкретному пользователю и участвует в механизмах разграничения доступа в других модулях системы.

4. Компонент «Модуль объявления» отвечает за каталог арендуемых вещей, категории, параметры объявлений и медиа-материалы. Данный модуль предоставляет данные для поиска и просмотра товаров, а также используется в сценариях оформления бронирования.

5. Компонент «Модуль бронирования» реализует обработку заявок на аренду, контроль статусов, проверку пересечения дат и платежное состояние сделки. На диаграмме отражено, что модуль бронирований связан с contracts и notifications: в первом случае — для юридического оформления аренды, во втором — для рассылки событий участникам сделки.

6. Компонент «Модуль договоров» отвечает за формирование договора аренды и фиксацию подписания сторонами (демонстрационная ЭЦП). Модуль получает данные из контура бронирований и возвращает признак подписанного документа, который влияет на дальнейшие этапы сделки.

7. Компонент «Модуль чатов» обеспечивает обмен сообщениями между участниками аренды в режиме реального времени. Согласно диаграмме, данный модуль связан с модулем уведомлений, что позволяет уведомлять пользователей о новых сообщениях.

8. Компонент «Модуль уведомлений» используется для формирования и хранения уведомлений о событиях системы: создании и изменении брони, подтверждении оплаты, новых сообщениях и отзывах. Компонент получает события от модуля бронирования и модуля чатов.

9. Компонент «PostgreSQL» является центральным хранилищем структурированных данных платформы. Через него сохраняются учетные записи, объявления, бронирования, договоры, сообщения и уведомления, обеспечивая целостность и согласованность состояния системы.

10. Компонент «Redis» применяется для поддержки событийного обмена в real-time контуре (Channels), прежде всего для WebSocket-сценариев. Это обеспечивает стабильную доставку сообщений и уведомлений в активные клиентские сессии.

11. Компонент «Медиа хранилище» предназначен для хранения файловых объектов системы: изображений товаров, медиа-вложений чатов и файлов договоров. Прикладной слой обращается к данному компоненту при загрузке и выдаче медиа-данных.

12. Компонент «SMTP» используется для отправки сервисных email-сообщений. Взаимодействие с почтовым каналом выполняется из серверного приложения через соответствующие сервисные функции.

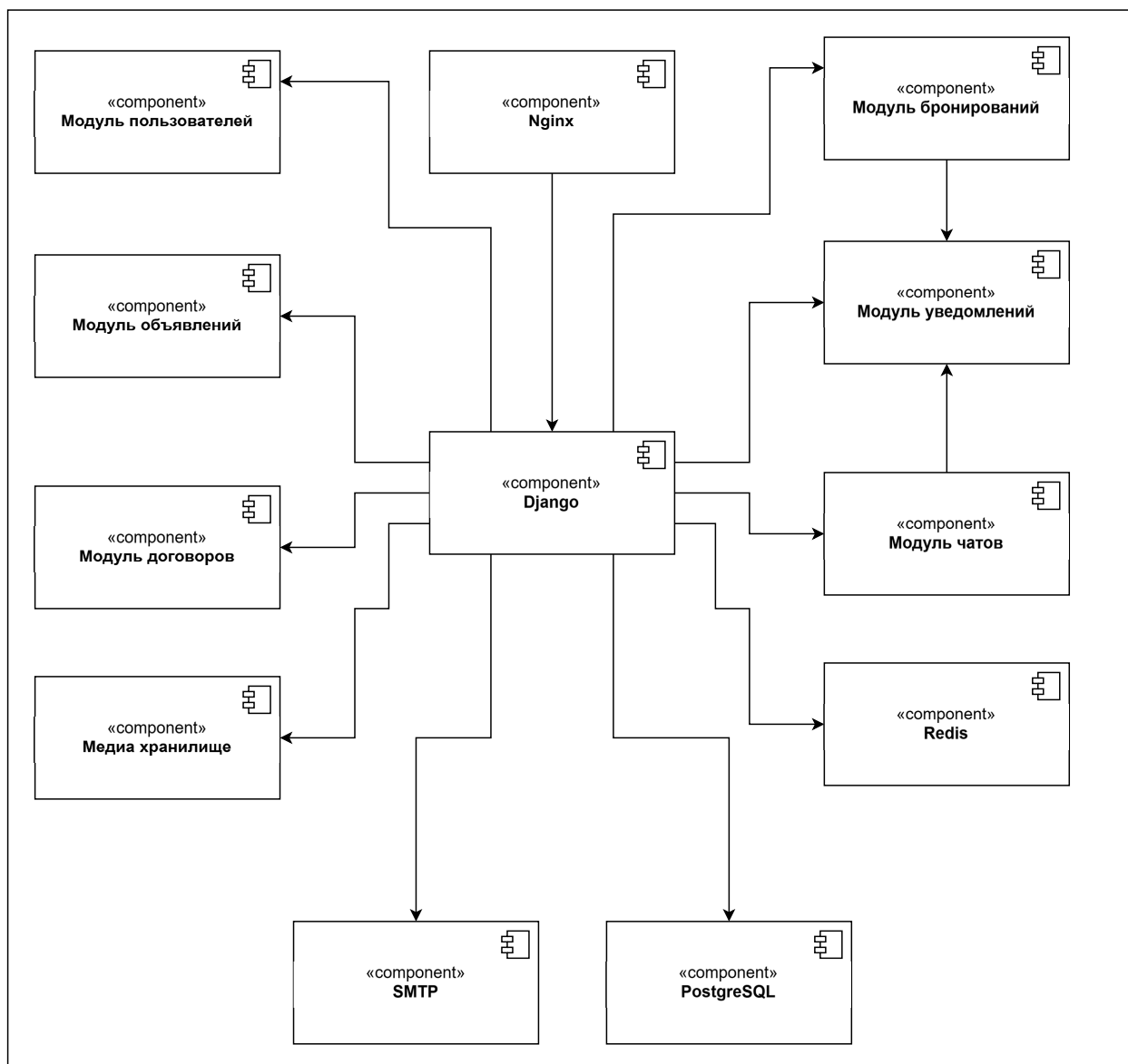


Рисунок 3.2 – Диаграмма компонентов

3.3.3 Маршрутизация API-endpoints

Серверная часть предоставляет REST API с единым префиксом /api. Аутентификация осуществляется с использованием JWT-токенов вида Authorization: Bearer <token>, передаваемых в заголовке. Для административных сценариев предусмотрен query-параметр, позволяющий получить полный набор записей (включая скрытые). В таблице ?? приведены основные API-endpoints серверной части.

Таблица 3.1 – Основные API-endpoints

Метод	Endpoint	Назначение
POST	/api/token/	Получение JWT-пары (access/refresh)
POST	/api/token/refresh/	Обновление access-токена
POST	/api/users/register/	Регистрация нового пользователя
POST	/api/users/send-email-code/	Отправка кода подтверждения на email
GET, PUT	/api/users/profile/	Просмотр и обновление профиля
POST	/api/users/change-password/	Смена пароля авторизованным пользователем
POST	/api/users/password-reset/request/	Запрос кода восстановления пароля
POST	/api/users/password-reset/confirm/	Подтверждение кода и установка нового пароля
POST	/api/users/fill-by-inn/	Автозаполнение реквизитов по ИНН
GET, POST	/api/items/categories/	Список и создание категорий
GET, POST	/api/items/	Список товаров и создание объявления
GET, PUT, PATCH, DELETE	/api/items/{id}/	Получение, изменение и удаление товара
GET	/api/items/{id}/booked_ranges/	Получение занятых диапазонов дат

Продолжение таблицы ??

Метод	Endpoint	Назначение
GET, POST	/api/items/{id}/reviews/	Список и добавление отзывов к товару
POST	/api/items/upload-image/	Загрузка изображения товара
DELETE	/api/items/images/{id}/	Удаление изображения товара
GET, POST	/api/items/moderation-requests/	Список заявок на модерацию и подача новой
GET, POST	/api/bookings/	Список бронирований и создание заявки
GET	/api/bookings/{id}/	Получение данных бронирования
POST	/api/bookings/{id}/confirm/	Подтверждение брони владельцем
POST	/api/bookings/{id}/cancel/	Отмена бронирования участником сделки
POST	/api/bookings/{id}/renter_cancel/	Отмена бронирования арендатором с указанием причины
POST	/api/bookings/{id}/complete/	Завершение аренды владельцем
POST	/api/bookings/{id}/start_payment/	Инициализация платёжной сессии
POST	/api/bookings/{id}/confirm_payment/	Подтверждение оплаты
POST	/api/bookings/{id}/confirm_renter_pickup/	Подтверждение получения товара арендатором

Продолжение таблицы ??

Метод	Endpoint	Назначение
POST	/api/bookings/{id} /confirm_owner_return/	Подтверждение возврата товара владельцем
POST	/api/bookings/{id} /return_deposit/	Возврат залога арендатору
POST	/api/bookings /{id}/request_refund/	Запрос возврата средств арендатором
POST	/api/bookings /{id}/process_refund/	Обработка возврата средств
POST	/api/bookings /{id}/review/	Оставить отзыв по завершённой бронированию
GET	/api/schema/	OpenAPI-схема
GET	/api/docs/	Интерактивная документация Swagger UI

Обмен данными между клиентом и сервером выполняется в формате JSON по протоколу HTTP. Для операций с файлами (загрузка изображений в каталог и фотографий при приёме/возврате товара) используется multipart/form-data.

Структура ответов соответствует практике DRF: при успешных операциях возвращаются сериализованные объекты предметной области (пользователь, товар, бронирование), а при ошибках — JSON-объект с диагностикой и HTTP-кодом (400, 403, 404 и др.). Для коллекций используется пагинация, что особенно важно для списков товаров и бронирований. Примеры ответов в формате JSON приведены ниже.

Пример успешного ответа POST /api/token/:

```
{
```

```

"refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJ0b2tldl90eXBlijoicmV
mcmVzaCI6ImV4cCI6MTc3ODkzNjUzMSwiaWF0IjoxNzc4ODU
wMTMxLCJqdGkiOiIyYWEzOTc3M2JkNmI
OMDZIYTBmMWU4NDcwMjE0Y2NjNSIsInVzZXJfaWQiOiIxIn
0.-NmgKonAW-zYhopagAP0-QL4MVdyaQ4f026njo21yQg",
"access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJ0b2tldl90eXBlijojYWNjZXNzIiwiaXhwIjoxN
zc4ODUzNzZlcjYXQiOiE3Nzg4NTAxMzEsImp0aS
I6IjBkMDM4ZGMzYmRINzQ4NWl4NGViYTY4MGVhMj
FmZmQzIiwidXNlcl9pZCI6IjEifQ.GH8jxpDq7jR2NCh10a7iUO29ZvP7ucLTNCQg6cPuz_8"
}

```

Пример успешного ответа POST /api/token/refresh/

```

{
  "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJ0b2tldl90eXBlijojYWNjZXNzIiwiaXhwIjoxNzc
4ODUzODE2LCJpYXQiOiE3Nzg4NTAyMTYsImp0aSI6Ij
c1ODgzZjI2OGUyYzQ5OWZhOTFjMjA2ZTM
wOWY3Y2E5IiwidXNlcl9pZCI6IjEifQ.IN
Q1_mKq3DCIhqYFuaHsq0c58vYooRF4bkzIa9Yb45s"
}

```

Пример успешного ответа POST /api/users/register/

```

{
  "username": "ermuk",
  "email": "user@gmail.com",
  "user_type": "individual",
  "full_name": "Иванов Иван Иванович",
  "entrepreneur_name": "",
  "company_name": "",
  "passport_series": "1111",
  "passport_number": "111111",
}

```

```
"inn": "111111111111",  
"kpp": "",  
"ogrnip": ""  
}
```

Пример успешного ответа POST /api/users/send-email-code/

```
{  
  "detail": "Код подтверждения отправлен на email."  
}
```

Пример успешного ответа GET /api/users/profile/

```
{  
  "username": "ermuk",  
  "email": "prectosic@gmail.com",  
  "user_type": "individual",  
  "full_name": "Иванов Иван Иванович",  
  "entrepreneur_name": "",  
  "company_name": "",  
  "passport_series": "1111",  
  "passport_number": "111111",  
  "inn": "111111111111",  
  "kpp": "",  
  "ogrnip": ""  
}
```

Пример успешного ответа PUT /api/users/profile/

```
{  
  "username": "ermuk",  
  "email": "prectosic@gmail.com",  
  "user_type": "individual",  
  "full_name": "Иванов Иван Иванович",  
  "entrepreneur_name": "",  
  "company_name": "",  
  "passport_series": "1111",
```

```
"passport_number": "111111",  
"inn": "111111111111",  
"kpp": "",  
"ogrnip": ""  
}
```

Пример успешного ответа POST /api/users/change-password/

```
{  
  "detail": "Пароль успешно изменён."  
}
```

Пример успешного ответа POST /api/users/password-reset/request/

```
{  
  "detail": "Код восстановления отправлен на email."  
}
```

Пример успешного ответа POST /api/users/password-reset/confirm/

```
{  
  "detail": "Пароль успешно сброшен."  
}
```

Пример успешного ответа POST /api/users/fill-by-inn/

```
{  
  "full_name": "Иванов Иван Иванович",  
  "company_name": "",  
  "entrepreneur_name": "ИП Иванов Иван Иванович",  
  "kpp": "",  
  "ogrnip": "123456789012345"  
}
```

Пример успешного ответа GET /api/items/categories/

```
{  
  "count": 6,  
  "next": null,  
  "previous": null,  
  "results": [  
    {  
      "id": 1,  
      "name": "Категория 1",  
      "description": "Описание категории 1",  
      "parent_id": null,  
      "is_active": true,  
      "created_at": "2023-01-01T00:00:00Z",  
      "updated_at": "2023-01-01T00:00:00Z"  
    },  
    {  
      "id": 2,  
      "name": "Категория 2",  
      "description": "Описание категории 2",  
      "parent_id": null,  
      "is_active": true,  
      "created_at": "2023-01-01T00:00:00Z",  
      "updated_at": "2023-01-01T00:00:00Z"  
    },  
    {  
      "id": 3,  
      "name": "Категория 3",  
      "description": "Описание категории 3",  
      "parent_id": null,  
      "is_active": true,  
      "created_at": "2023-01-01T00:00:00Z",  
      "updated_at": "2023-01-01T00:00:00Z"  
    },  
    {  
      "id": 4,  
      "name": "Категория 4",  
      "description": "Описание категории 4",  
      "parent_id": null,  
      "is_active": true,  
      "created_at": "2023-01-01T00:00:00Z",  
      "updated_at": "2023-01-01T00:00:00Z"  
    },  
    {  
      "id": 5,  
      "name": "Категория 5",  
      "description": "Описание категории 5",  
      "parent_id": null,  
      "is_active": true,  
      "created_at": "2023-01-01T00:00:00Z",  
      "updated_at": "2023-01-01T00:00:00Z"  
    },  
    {  
      "id": 6,  
      "name": "Категория 6",  
      "description": "Описание категории 6",  
      "parent_id": null,  
      "is_active": true,  
      "created_at": "2023-01-01T00:00:00Z",  
      "updated_at": "2023-01-01T00:00:00Z"  
    }  
  ]  
}
```

```

    { "id": 1, "name": "string" },
    { "id": 6, "name": "Дом и сад" },
    { "id": 3, "name": "Инструменты" },
    { "id": 4, "name": "Одежда" },
    { "id": 5, "name": "Спорт" },
    { "id": 2, "name": "Электроника" }
  ]
}

```

Пример успешного ответа POST /api/items/categories/

```

{
  "id": 7,
  "name": "Крепёж"
}

```

Пример успешного ответа GET /api/items/

```

{
  "count": 2,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 2,
      "images": [
        {
          "id": 2,
          "image": "http://localhost/media/items/bd1294c75f7a56651676df8759d14e79.jpg"
        }
      ],
      "booked_ranges": [],
      "owner_username": "ermuk",
      "item_reviews": [],
      "owner_rating": "0.00",

```

```

    "owner_reviews_count": 0,
    "title": "aaa",
    "description": "aaa",
    "price_per_day": "111.00",
    "average_rating": "0.00",
    "reviews_count": 0,
    "status": "available",
    "created_at": "2026-05-13T11:55:30.385864Z",
    "owner": 1,
    "category": null
  }
]
}

```

Пример успешного ответа POST /api/items/

```

{
  "id": 3,
  "images": [],
  "booked_ranges": [],
  "owner_username": "ermuk",
  "category_name": "string",
  "item_reviews": [],
  "owner_rating": "0.00",
  "owner_reviews_count": 0,
  "title": "тест",
  "description": "тест",
  "price_per_day": "81.00",
  "average_rating": "0.00",
  "reviews_count": 0,
  "status": "available",
  "created_at": "2026-05-15T13:33:19.355414Z",
  "owner": 1,

```

```
"category": 1
}
```

Пример успешного ответа GET /api/items/{id}/

```
{
  "id": 3,
  "images": [],
  "booked_ranges": [],
  "owner_username": "ermuk",
  "category_name": "string",
  "item_reviews": [],
  "owner_rating": "0.00",
  "owner_reviews_count": 0,
  "title": "тест",
  "description": "тест",
  "price_per_day": "81.00",
  "average_rating": "0.00",
  "reviews_count": 0,
  "status": "available",
  "created_at": "2026-05-15T13:33:19.355414Z",
  "owner": 1,
  "category": 1
}
```

Пример успешного ответа PUT /api/items/{id}/

```
{
  "id": 3,
  "images": [],
  "booked_ranges": [],
  "owner_username": "ermuk",
  "category_name": "string",
  "item_reviews": [],
  "owner_rating": "0.00",
```

```
"owner_reviews_count": 0,  
"title": "тест",  
"description": "тест",  
"price_per_day": "81.00",  
"average_rating": "0.00",  
"reviews_count": 0,  
"status": "available",  
"created_at": "2026-05-15T13:33:19.355414Z",  
"owner": 1,  
"category": 1  
}
```

Пример успешного ответа PATCH /api/items/{id}/

```
{  
  "id": 3,  
  "images": [],  
  "booked_ranges": [],  
  "owner_username": "ermuk",  
  "category_name": "string",  
  "item_reviews": [],  
  "owner_rating": "0.00",  
  "owner_reviews_count": 0,  
  "title": "тест",  
  "description": "тест",  
  "price_per_day": "81.00",  
  "average_rating": "0.00",  
  "reviews_count": 0,  
  "status": "available",  
  "created_at": "2026-05-15T13:33:19.355414Z",  
  "owner": 1,  
  "category": 1  
}
```

Пример успешного ответа DELETE /api/items/{id}/

```
{  
  "detail": ""  
}
```

Пример успешного ответа GET /api/items/{id}/booked_ranges/

```
[  
  {  
    "start_date": "2026-05-13",  
    "end_date": "2026-05-14"  
  },  
  {  
    "start_date": "2026-05-17",  
    "end_date": "2026-05-18"  
  }  
]
```

Пример успешного ответа GET /api/items/{id}/reviews/

```
{  
  "count": 2,  
  "next": null,  
  "previous": null,  
  "results": [  
    {  
      "id": 12,  
      "item": 5,  
      "author": 7,  
      "author_username": "ivan",  
      "rating": 5,  
      "comment": "Отличное объявление",  
      "created_at": "2026-05-15T12:34:56.789012+03:00"  
    },  
    {
```

```
"id": 11,  
"item": 5,  
"author": 8,  
"author_username": "olga",  
"rating": 4,  
"comment": "Все хорошо",  
"created_at": "2026-05-14T10:20:30.123456+03:00"  
}  
]  
}
```

Пример успешного ответа POST /api/items/{id}/reviews/

```
{  
"id": 12,  
"item": 5,  
"author": 7,  
"author_username": "ivan",  
"rating": 5,  
"comment": "Отличное объявление",  
"created_at": "2026-05-15T12:34:56.789012+03:00"  
}
```

Пример успешного ответа POST /api/items/upload-image/

```
{  
"id": 3,  
"image": "/media/items/test.jpg"  
}
```

Пример успешного ответа DELETE /api/items/images/{id}/

```
{  
"detail": "Изображение удалено."  
}
```

Пример успешного ответа GET /api/items/moderation-requests/

```
{
```

```
"count": 1,  
"next": null,  
"previous": null,  
"results": [  
  {  
    "id": 1,  
    "item": 3,  
    "item_title": "Перфоратор Bosch",  
    "status": "pending",  
    "created_at": "2026-05-15T10:00:00.000000Z"  
  }  
]  
}
```

Пример успешного ответа POST /api/items/moderation-requests/

```
{  
  "id": 2,  
  "item": 4,  
  "item_title": "Шуруповёрт Makita",  
  "status": "pending",  
  "created_at": "2026-05-20T09:15:00.000000Z"  
}
```

Пример успешного ответа GET /api/bookings/

```
{  
  "count": 2,  
  "next": null,  
  "previous": null,  
  "results": [  
    {  
      "id": 2,  
      "item_title": "aaa",  
      "renter_username": "Admin",
```

```

    "renter_email": "enot.valentin@yandex.ru",
    "has_review": false,
    "can_leave_review": true,
    "review": null,
    "start_date": "2026-05-17",
    "end_date": "2026-05-18",
    "total_price": "111.00",
    "status": "confirmed",
    "payment_method": "sbp",
    "payment_status": "paid",
    "payment_reference": "ecae5239-8012-4fbb-a45d-49572ee1e504",
    "payment_expires_at": null,
    "paid_at": "2026-05-15T15:04:38.034157Z",
    "created_at": "2026-05-13T11:28:43.708629Z",
    "item": 1,
    "renter": 2
  }
]
}

```

Пример успешного ответа POST /api/bookings/

```

{
  "id": 3,
  "item_title": "aaa",
  "renter_username": "Admin",
  "renter_email": "enot.valentin@yandex.ru",
  "has_review": false,
  "can_leave_review": false,
  "review": null,
  "start_date": "2026-05-15",
  "end_date": "2026-05-16",
  "total_price": "111.00",

```

```
"status": "pending",
"payment_method": "sbp",
"payment_status": "unpaid",
"payment_reference": null,
"payment_expires_at": null,
"paid_at": null,
"created_at": "2026-05-15T15:08:38.941316Z",
"item": 1,
"renter": 2
}
```

Пример успешного ответа GET /api/bookings/{id}/

```
{
  "id": 3,
  "item_title": "aaa",
  "renter_username": "Admin",
  "renter_email": "enot.valentin@yandex.ru",
  "has_review": false,
  "can_leave_review": false,
  "review": null,
  "start_date": "2026-05-15",
  "end_date": "2026-05-16",
  "total_price": "111.00",
  "status": "pending",
  "payment_method": "sbp",
  "payment_status": "unpaid",
  "payment_reference": null,
  "payment_expires_at": null,
  "paid_at": null,
  "created_at": "2026-05-15T15:08:38.941316Z",
  "item": 1,
  "renter": 2
}
```

}

Пример успешного ответа POST /api/bookings/{id}/confirm/

{

```
"id": 3,  
"item_title": "aaa",  
"renter_username": "Admin",  
"renter_email": "enot.valentin@yandex.ru",  
"has_review": false,  
"can_leave_review": false,  
"review": null,  
"start_date": "2026-05-15",  
"end_date": "2026-05-16",  
"total_price": "111.00",  
"status": "confirmed",  
"payment_method": "sbp",  
"payment_status": "unpaid",  
"payment_reference": null,  
"payment_expires_at": null,  
"paid_at": null,  
"created_at": "2026-05-15T15:08:38.941316Z",  
"item": 1,  
"renter": 2
```

}

Пример успешного ответа POST /api/bookings/{id}/cancel/

{

```
"id": 3,  
"item_title": "aaa",  
"renter_username": "Admin",  
"renter_email": "enot.valentin@yandex.ru",  
"has_review": false,  
"can_leave_review": false,
```

```
"review": null,  
"start_date": "2026-05-15",  
"end_date": "2026-05-16",  
"total_price": "111.00",  
"status": "cancelled",  
"payment_method": "sbp",  
"payment_status": "unpaid",  
"payment_reference": null,  
"payment_expires_at": null,  
"paid_at": null,  
"created_at": "2026-05-15T15:08:38.941316Z",  
"item": 1,  
"renter": 2  
}
```

Пример успешного ответа POST /api/bookings/{id}/renter_cancel/

```
{  
"id": 3,  
"item_title": "aaa",  
"renter_username": "Admin",  
"renter_email": "enot.valentin@yandex.ru",  
"has_review": false,  
"can_leave_review": false,  
"review": null,  
"start_date": "2026-05-15",  
"end_date": "2026-05-16",  
"total_price": "111.00",  
"status": "cancelled",  
"payment_method": "sbp",  
"payment_status": "refunded",  
"payment_reference": null,  
"payment_expires_at": null,  
}
```

```
"paid_at": null,  
"renter_cancel_reason": "Изменились планы",  
"created_at": "2026-05-15T15:08:38.941316Z",  
"item": 1,  
"renter": 2  
}
```

Пример успешного ответа POST /api/bookings/{id}/complete/

```
{  
  "id": 4,  
  "item_title": "aaa",  
  "renter_username": "Admin",  
  "renter_email": "enot.valentin@yandex.ru",  
  "has_review": false,  
  "can_leave_review": false,  
  "review": null,  
  "start_date": "2026-05-15",  
  "end_date": "2026-05-16",  
  "total_price": "111.00",  
  "status": "completed",  
  "payment_method": "sbp",  
  "payment_status": "paid",  
  "payment_reference": null,  
  "payment_expires_at": null,  
  "paid_at": null,  
  "created_at": "2026-05-15T15:11:29.438091Z",  
  "item": 1,  
  "renter": 2  
}
```

Пример успешного ответа POST /api/bookings/{id}/start_payment/

```
{  
  "id": 4,
```

```
"item_title": "aaa",
"renter_username": "Admin",
"renter_email": "enot.valentin@yandex.ru",
"has_review": false,
"can_leave_review": false,
"review": null,
"start_date": "2026-05-15",
"end_date": "2026-05-16",
"total_price": "111.00",
"status": "confirmed",
"payment_method": "sbp",
"payment_status": "pending",
"payment_reference": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
"payment_expires_at": "2026-05-15T15:11:29.438091Z",
"paid_at": null,
"created_at": "2026-05-15T15:11:29.438091Z",
"item": 1,
"renter": 2
}
```

Пример успешного ответа POST /api/bookings/{id}/confirm_payment/

```
{
  "id": 4,
  "item_title": "aaa",
  "renter_username": "Admin",
  "renter_email": "enot.valentin@yandex.ru",
  "has_review": false,
  "can_leave_review": false,
  "review": null,
  "start_date": "2026-05-15",
  "end_date": "2026-05-16",
  "total_price": "111.00",
```

```
"status": "completed",
"payment_method": "sbp",
"payment_status": "paid",
"payment_reference": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
"payment_expires_at": "2026-05-15T15:11:29.438091Z",
"paid_at": null,
"created_at": "2026-05-15T15:11:29.438091Z",
"item": 1,
"renter": 2
}
```

Пример успешного ответа POST /api/bookings/{id}/confirm_renter_pickup/

```
{
  "id": 4,
  "item_title": "aaa",
  "renter_username": "Admin",
  "renter_email": "enot.valentin@yandex.ru",
  "has_review": false,
  "can_leave_review": false,
  "review": null,
  "start_date": "2026-05-15",
  "end_date": "2026-05-16",
  "total_price": "111.00",
  "status": "confirmed",
  "payment_method": "sbp",
  "payment_status": "paid",
  "renter_pickup_confirmed": true,
  "renter_pickup_confirmed_at": "2026-05-15T16:00:00.000000Z",
  "renter_pickup_photo": "/media/bookings/pickup_4.jpg",
  "owner_return_confirmed": false,
  "owner_return_confirmed_at": null,
  "owner_return_photo": null,
}
```

```
"created_at": "2026-05-15T15:11:29.438091Z",  
"item": 1,  
"renter": 2  
}
```

Пример успешного ответа POST /api/bookings/{id}/confirm_owner_return/

```
{  
  "id": 4,  
  "item_title": "aaa",  
  "renter_username": "Admin",  
  "renter_email": "enot.valentin@yandex.ru",  
  "has_review": false,  
  "can_leave_review": true,  
  "review": null,  
  "start_date": "2026-05-15",  
  "end_date": "2026-05-16",  
  "total_price": "111.00",  
  "status": "completed",  
  "payment_method": "sbp",  
  "payment_status": "paid",  
  "renter_pickup_confirmed": true,  
  "renter_pickup_confirmed_at": "2026-05-15T16:00:00.000000Z",  
  "renter_pickup_photo": "/media/bookings/pickup_4.jpg",  
  "owner_return_confirmed": true,  
  "owner_return_confirmed_at": "2026-05-16T12:00:00.000000Z",  
  "owner_return_photo": "/media/bookings/return_4.jpg",  
  "created_at": "2026-05-15T15:11:29.438091Z",  
  "item": 1,  
  "renter": 2  
}
```

Пример успешного ответа POST /api/bookings/{id}/return_deposit/

```
{
```

```
"id": 4,  
"item_title": "aaa",  
"renter_username": "Admin",  
"renter_email": "enot.valentin@yandex.ru",  
"has_review": false,  
"can_leave_review": true,  
"review": null,  
"start_date": "2026-05-15",  
"end_date": "2026-05-16",  
"total_price": "111.00",  
"status": "completed",  
"payment_method": "sbp",  
"payment_status": "paid",  
"deposit_amount": "500.00",  
"deposit_status": "returned",  
"created_at": "2026-05-15T15:11:29.438091Z",  
"item": 1,  
"renter": 2  
}
```

Пример успешного ответа POST /api/bookings/{id}/request_refund/

```
{  
"id": 4,  
"item_title": "aaa",  
"renter_username": "Admin",  
"renter_email": "enot.valentin@yandex.ru",  
"has_review": false,  
"can_leave_review": false,  
"review": null,  
"start_date": "2026-05-15",  
"end_date": "2026-05-16",  
"total_price": "111.00",
```

```
"status": "confirmed",
"payment_method": "sbp",
"payment_status": "paid",
"return_requested": true,
"return_requested_at": "2026-05-16T10:00:00.000000Z",
"return_reason": "Товар не соответствует описанию",
"return_status": "pending",
"created_at": "2026-05-15T15:11:29.438091Z",
"item": 1,
"renter": 2
}
```

Пример успешного ответа POST /api/bookings/{id}/process_refund/

```
{
  "id": 4,
  "item_title": "aaa",
  "renter_username": "Admin",
  "renter_email": "enot.valentin@yandex.ru",
  "has_review": false,
  "can_leave_review": false,
  "review": null,
  "start_date": "2026-05-15",
  "end_date": "2026-05-16",
  "total_price": "111.00",
  "status": "cancelled",
  "payment_method": "sbp",
  "payment_status": "refunded",
  "return_requested": true,
  "return_status": "approved",
  "created_at": "2026-05-15T15:11:29.438091Z",
  "item": 1,
  "renter": 2
}
```

```
}
```

Пример успешного ответа POST /api/bookings/{id}/review/

```
{
```

```
"id": 7,
```

```
"booking": 4,
```

```
"item_id": 1,
```

```
"rating": 5,
```

```
"comment": "Всё отлично, вещь в хорошем состоянии.",
```

```
"created_at": "2026-05-20T10:15:00.000000Z",
```

```
"author_username": "Admin"
```

```
}
```

Пример успешного ответа GET /api/schema/

```
{
```

```
"openapi": "3.0.3",
```

```
"info": {
```

```
  "title": "Arenda API",
```

```
  "version": "1.0.0",
```

```
  "description": "API для онлайн площадки аренды вещей"
```

```
}
```

```
}
```

3.4 Модель данных

Сущности и отношение между таблицами базы данных отражены на рисунке ??.



Рисунок 3.3 – ER-диаграмма

3.4.1 Нормализация схемы данных

Схема данных приведена к третьей нормализованной форме. Повторяющиеся группы данных вынесены в отдельные таблицы: категории отделены от товаров, медиа хранится отдельно от карточки товара, сообщения отделены от чатов. Это устраняет избыточность и снижает риск аномалий при обновлении данных. Зависимости между полями организованы по принципу «атрибут зависит от ключа и только от ключа».

3.4.2 Описание структуры таблиц

В таблице ?? представлена структура таблицы User. Данная таблица содержит данные о пользователях платформы: уникальный идентификатор, имя пользователя, адрес электронной почты, пароль, тип пользователя (физическое лицо, ИП, юридическое лицо), номер телефона, ФИО, наименование

ИП или компании, паспортные реквизиты, ИНН, КПП, ОГРНИП, признак за-
полненности профиля, средний рейтинг и количество отзывов.

Таблица 3.2 – Таблица User

Тип ключа	Имя столбца	Тип данных	Обязательность
primary key	id	Integer	true
	username	Text	true
	email	Text	true
	password	Text	true
	user_type	Text	true
	phone	Text	false
	full_name	Text	false
	entrepreneur_name	Text	false
	company_name	Text	false
	passport_series	Text	false
	passport_number	Text	false
	inn	Text	false
	kpp	Text	false
	ogrnip	Text	false
	profile_completed	Boolean	true
	average_rating	Decimal	true
	reviews_count	Integer	true

В таблице ?? представлена структура таблицы Category. Данная таб-
лица содержит данные о категориях товаров: уникальный идентификатор и
наименование категории.

Таблица 3.3 – Таблица Category

Тип ключа	Имя столбца	Тип данных	Обязательность
primary key	id	Integer	true

Продолжение таблицы ??

Тип ключа	Имя столбца	Тип данных	Обязательность
	name	Text	true

В таблице ?? представлена структура таблицы Item. Данная таблица содержит данные о товарах, размещаемых для аренды: уникальный идентификатор, идентификатор владельца, идентификатор категории, название, описание, стоимость аренды за сутки, размер залога, статус доступности, средний рейтинг, количество отзывов и дату создания записи.

Таблица 3.4 – Таблица Item

Тип ключа	Имя столбца	Тип данных	Обязательность
primary key	id	Integer	true
foreign key	owner_id	Integer	true
foreign key	category_id	Integer	false
	title	Text	true
	description	Text	true
	price_per_day	Decimal	true
	deposit	Decimal	true
	status	Text	true
	average_rating	Decimal	true
	reviews_count	Integer	true
	created_at	DateTime	true

В таблице ?? представлена структура таблицы ItemImage. Данная таблица содержит данные об изображениях товаров: уникальный идентификатор, идентификатор товара, путь к файлу изображения и дату загрузки.

Таблица 3.5 – Таблица ItemImage

Тип ключа	Имя столбца	Тип данных	Обязательность
primary key	id	Integer	true
foreign key	item_id	Integer	true
	image	Text	true
	uploaded_at	DateTime	true

В таблице ?? представлена структура таблицы Booking. Данная таблица содержит данные о бронированиях: уникальный идентификатор, идентификатор товара, идентификатор арендатора, дату начала и окончания аренды, итоговую стоимость, сумму аренды без залога и комиссии, сервисный сбор платформы, статус бронирования, способ оплаты, статус оплаты, платёжный идентификатор, срок действия платёжной сессии, время подтверждения оплаты, данные залога (сумма, статус, идентификатор, срок сессии, время оплаты), данные о подтверждении получения товара арендатором и возврата товара арендодателем (с фотографиями), данные о запросе возврата средств, причину отказа арендатора и дату создания бронирования.

Таблица 3.6 – Таблица Booking

Тип ключа	Имя столбца	Тип данных	Обязательность
primary key	id	Integer	true
foreign key	item_id	Integer	true
foreign key	renter_id	Integer	true
	start_date	Date	true
	end_date	Date	true
	total_price	Decimal	false
	rent_amount	Decimal	false
	platform_fee	Decimal	false
	status	Text	true
	payment_method	Text	true

Продолжение таблицы ??

Тип ключа	Имя столбца	Тип данных	Обязательность
	payment_status	Text	true
	payment_reference	Text	false
	payment_expires_at	DateTime	false
	paid_at	DateTime	false
	deposit_amount	Decimal	false
	deposit_status	Text	false
	deposit_reference	Text	false
	deposit_expires_at	DateTime	false
	deposit_paid_at	DateTime	false
	renter_pickup_confirmed	Boolean	true
	renter_pickup_confirmed_at	DateTime	false
	renter_pickup_photo	Text	false
	owner_return_confirmed	Boolean	true
	owner_return_confirmed_at	DateTime	false
	owner_return_photo	Text	false
	return_requested	Boolean	true
	return_requested_at	DateTime	false
	return_reason	Text	false
	return_status	Text	false
	renter_cancel_reason	Text	false
	created_at	DateTime	true

В таблице ?? представлена структура таблицы Contract. Данная таблица содержит данные о договорах аренды: уникальный идентификатор, идентификатор бронирования, номер договора, путь к файлу договора, признак подписания, данные подписи арендатора (ФИО подписанта, код подписи, дата и время подписи), данные подписи владельца (ФИО подписанта, код под-

писи, дата и время подписи), общее время подписания договора и дату создания записи.

Таблица 3.7 – Таблица Contract

Тип ключа	Имя столбца	Тип данных	Обязательность
primary key	id	Integer	true
foreign key	booking_id	Integer	true
	document_number	Text	true
	file	Text	false
	is_signed	Boolean	true
	renter_signer_name	Text	false
	renter_signature_code	Text	false
	renter_signed_at	DateTime	false
	owner_signer_name	Text	false
	owner_signature_code	Text	false
	owner_signed_at	DateTime	false
	signed_at	DateTime	false
	created_at	DateTime	true

В таблице ?? представлена структура таблицы Review. Данная таблица содержит данные об отзывах на товары: уникальный идентификатор, идентификатор товара, идентификатор автора отзыва, оценку, комментарий и дату создания.

Таблица 3.8 – Таблица Review

Тип ключа	Имя столбца	Тип данных	Обязательность
primary key	id	Integer	true
foreign key	item_id	Integer	true
foreign key	author_id	Integer	true
	rating	Integer	true

Продолжение таблицы ??

Тип ключа	Имя столбца	Тип данных	Обязательность
	comment	Text	false
	created_at	DateTime	true

В таблице ?? представлена структура таблицы Chat. Данная таблица содержит данные о чатах: уникальный идентификатор, идентификатор связанного товара (при наличии) и дату создания чата.

Таблица 3.9 – Таблица Chat

Тип ключа	Имя столбца	Тип данных	Обязательность
primary key	id	Integer	true
foreign key	item_id	Integer	false
	created_at	DateTime	true

В таблице ?? представлена структура таблицы Message. Данная таблица содержит данные о сообщениях в чатах: уникальный идентификатор, идентификатор чата, идентификатор отправителя, текст сообщения, путь к изображению, дату и время отправки, а также признак прочтения.

Таблица 3.10 – Таблица Message

Тип ключа	Имя столбца	Тип данных	Обязательность
primary key	id	Integer	true
foreign key	chat_id	Integer	true
foreign key	sender_id	Integer	true
	text	Text	false
	image	Text	false
	created_at	DateTime	true
	is_read	Boolean	true

3.5 Реализация бизнес-логики

Бизнес-логика системы реализована на сервере и охватывает полный цикл аренды: публикацию товара, бронирование, подписание договора, оплаты и отзыв. Основные проверки выполняются в backend, что исключает зависимость корректности процесса от клиентской части.

3.5.1 Управление товарами

Модуль объявлений реализует CRUD-операции для объявлений. Создание и изменение товара доступны только авторизованному владельцу, а проверка прав выполняется на уровне permission-классов. Это предотвращает редактирование чужих объявлений и подмену владельца через запрос.

Загрузка фотографий вынесена в отдельный endpoint. Перед сохранением сервер проверяет существование товара и его принадлежность текущему пользователю.

3.5.2 Бронирование и проверка конфликтов дат

При создании бронирования сервер проверяет, что пользователь не арендует собственную вещь, а также корректность диапазона дат. Конфликт дат определяется по условию пересечения интервалов с активными бронированиями, что исключает двойную бронь.

Стоимость фиксируется в момент создания заявки и не зависит от последующего изменения цены товара. Для интерфейса календаря реализован endpoint занятых диапазонов, чтобы клиент мог сразу блокировать недоступные даты.

Договор считается полностью подписанным только после двух подписей. Этот статус используется как обязательное условие для перехода к оплате.

3.5.3 Платёжный модуль

Оплата реализована как демонстрационный сценарий: создается платежная сессия с идентификатором и временем жизни, а после подтверждения статус оплаты меняется на подтвержденный. Оплата доступна только для подтвержденной брони и только после полного подписания договора.

3.5.4 Система отзывов и рейтинга

Отзыв может оставить только арендатор по оплаченной или завершенной сделке. После добавления (или скрытия/удаления) отзыва сервер автоматически пересчитывает средний рейтинг и количество отзывов для товара и владельца. Это поддерживает актуальную репутационную модель и повышает доверие к платформе.

3.6 Информационная безопасность

Серверная часть веб-приложения обрабатывает персональные данные, сведения о бронированиях, платежные статусы и переписку пользователей, поэтому требования безопасности являются критичными. Основные риски связаны с несанкционированным доступом к операциям, подменой параметров запросов, попытками изменения чужих данных и нарушением логики бизнес-процессов.

В проекте защита строится на JWT-аутентификации, проверках прав доступа и серверной валидации бизнес-правил. Изменять объявления может только владелец. Дополнительно реализованы проверки дат, запрет бронирования собственной вещи и контроль пересечений периодов аренды.

3.6.1 Профиль угроз

Профиль угроз включает:

- несанкционированный доступ к API и данным пользователей;
- подмену бизнес-операций (статусы, чужие ресурсы);
- компрометацию токенов и учетных данных;

- эксплуатационные риски небезопасной конфигурации.

3.6.2 Меры защиты

Меры защиты включают JWT, permission-политику, валидацию входных данных и ограничение бизнес-переходов на сервере. Для договоров применяется двустороннее подписание, а оплата разрешена только после выполнения необходимых условий сделки.

3.7 Эксплуатация и развёртывание

Развёртывание выполнено в контейнерной архитектуре: Django-приложение, PostgreSQL, Redis и Nginx запускаются через Docker Compose. Такой подход обеспечивает воспроизводимость окружения и упрощает сопровождение. Параметры конфигурации (переменные окружения) вынесены в окружение:

- DEBUG;
- SECRET-KEY;
- параметры БД;
- SMTP-данные.

Типовой сценарий развёртывания:

1. Запуск контейнеров через Docker Compose.
2. Создание и применение миграций.
3. Создание администратора и проверка доступности API.

4 Рабочий проект

В процессе разработки серверная часть была реализована в виде REST API на базе Django и Django REST Framework. Проект разделён на следующие модули: users, items, bookings, arenda_site.

4.1 Модуль users

Модуль users реализует регистрацию, аутентификацию и управление профилем пользователей платформы.

4.1.1 Класс SendEmailVerificationCodeView

SendEmailVerificationCodeView — класс для отправки кода подтверждения email. Генерирует шестизначный код, сохраняет его в кэше и отправляет пользователю перед регистрацией.

Описание методов класса SendEmailVerificationCodeView представлено в таблице ??.

Таблица 4.1 – Описание методов класса SendEmailVerificationCodeView

Название метода	Аргументы метода	Описание метода
1	2	3
post	request	Принимает email, проверяет его уникальность. Генерирует шестизначный код, сохраняет в кэше на 15 минут и отправляет на указанный адрес. Возвращает ошибку, если email уже зарегистрирован

4.1.2 Класс RegisterView

RegisterView — класс для регистрации нового пользователя. Наследует CreateAPIView и расширяет его проверкой кода подтверждения email.

Описание методов класса RegisterView представлено в таблице ??.

Таблица 4.2 – Описание методов класса RegisterView

Название метода	Аргументы метода	Описание метода
1	2	3
create	request, *args, **kwargs	Проверяет наличие и корректность кода подтверждения email_code из кэша. При успехе создаёт нового пользователя и удаляет код из кэша. При отсутствии или несовпадении кода возвращает HTTP 400

4.1.3 Класс FillByInnView

FillByInnView — класс для автозаполнения реквизитов по ИНН. Обращается к внешнему сервису DaData и возвращает данные юридического лица или ИП. Доступен без аутентификации.

Описание методов класса FillByInnView представлено в таблице ??.

Таблица 4.3 – Описание методов класса FillByInnView

Название метода	Аргументы метода	Описание метода
1	2	3
get	request	Принимает параметр inn из строки запроса. Обращается к DaData и возвращает реквизиты: для юридического лица — ИНН, КПП, наименование; для ИП — ИНН, ОГРНИП, наименование

4.1.4 Класс ProfileView

ProfileView — класс для просмотра и редактирования профиля текущего пользователя. Требуется аутентификация.

Описание методов класса ProfileView представлено в таблице ??.

Таблица 4.4 – Описание методов класса ProfileView

Название метода	Аргументы метода	Описание метода
1	2	3
get_object	—	Возвращает объект текущего аутентифицированного пользователя
get	request	Возвращает сериализованные данные профиля текущего пользователя
put	request	Обновляет данные профиля. При заполнении всех обязательных полей устанавливает флаг profile_completed = True

4.1.5 Класс ChangePasswordView

ChangePasswordView — класс для смены пароля аутентифицированного пользователя.

Описание методов класса ChangePasswordView представлено в таблице ??.

Таблица 4.5 – Описание методов класса ChangePasswordView

Название метода	Аргументы метода	Описание метода
1	2	3
post	request	Проверяет корректность текущего пароля (old_password). При совпадении устанавливает новый пароль (new_password) и сохраняет пользователя

4.1.6 Класс PasswordResetRequestView

PasswordResetRequestView — класс для запроса сброса пароля. Генерирует код сброса и отправляет его на email пользователя.

Описание методов класса PasswordResetRequestView представлено в таблице ??.

Таблица 4.6 – Описание методов класса PasswordResetRequestView

Название метода	Аргументы метода	Описание метода
1	2	3
post	request	Ищет пользователя по переданному email. Генерирует шестизначный код, сохраняет его в кэше на 15 минут и отправляет на почту

4.1.7 Класс PasswordResetConfirmView

PasswordResetConfirmView — класс для подтверждения сброса пароля по коду из письма.

Описание методов класса PasswordResetConfirmView представлено в таблице ??.

Таблица 4.7 – Описание методов класса PasswordResetConfirmView

Название метода	Аргументы метода	Описание метода
1	2	3
post	request	Принимает email, code и new_password. Сверяет код с кэшем, устанавливает новый пароль (не менее 6 символов) и удаляет код из кэша

4.2 Модуль items

Модуль items реализует создание и управление объявлениями об аренде, загрузку медиафайлов, категоризацию и систему модерации.

4.2.1 Класс CategoryViewSet

CategoryViewSet — класс для работы с категориями объявлений. Реализует стандартные операции чтения на основе ModelViewSet. Категории упорядочены по названию.

Описание методов класса CategoryViewSet представлено в таблице ??.

Таблица 4.8 – Описание методов класса CategoryViewSet

Название метода	Аргументы метода	Описание метода
1	2	3
list	request	Возвращает список всех категорий, упорядоченных по полю name
retrieve	request, pk	Возвращает категорию по идентификатору

4.2.2 Класс ItemViewSet

ItemViewSet — основной класс для управления объявлениями. Поддерживает фильтрацию по категории и статусу, поиск по заголовку и описанию, сортировку по цене, дате и рейтингу. Все создание и изменение объявлений автоматически направляются на модерацию.

Описание методов класса ItemViewSet представлено в таблице ??.

Таблица 4.9 – Описание методов класса ItemViewSet

Название метода	Аргументы метода	Описание метода
1	2	3
get_queryset	—	Возвращает список объявлений: для аутентифицированного пользователя — доступные и его собственные; для анонимного — только со статусом available. Поддерживает параметры mine и min_rating

Продолжение таблицы ??

1	2	3
get_permissions	—	Назначает права доступа в зависимости от действия: создание требует заполненного профиля (IsProfileCompleted), редактирование и удаление — владения объявлением (IsOwner)
perform_create	serializer	Сохраняет объявление со статусом pending и создаёт заявку на модерацию с типом create
perform_update	serializer	Сохраняет изменения, переводит объявление в статус pending и создаёт заявку на модерацию с типом update
booked_ranges	request, pk	Возвращает список занятых диапазонов дат на основе активных бронирований со статусом pending или confirmed
reviews	request, pk	GET — возвращает список видимых отзывов на объявление. POST — создаёт новый отзыв

4.2.3 Класс ItemImageUploadView

ItemImageUploadView — класс для загрузки изображений к объявлению. Использует парсеры MultiPartParser и FormParser для обработки загружаемых файлов. Требуется аутентификация.

Описание методов класса ItemImageUploadView представлено в таблице ??.

Таблица 4.10 – Описание методов класса ItemImageUploadView

Название метода	Аргументы метода	Описание метода
1	2	3
perform_create	serializer	Сохраняет загруженное изображение, привязывая его к объявлению текущего пользователя

4.2.4 Класс ItemImageDeleteView

ItemImageDeleteView — класс для удаления изображений объявления. Ограничивает доступ: пользователь может удалять только изображения своих объявлений.

Описание методов класса ItemImageDeleteView представлено в таблице ??.

Таблица 4.11 – Описание методов класса ItemImageDeleteView

Название метода	Аргументы метода	Описание метода
1	2	3
get_queryset	—	Возвращает только изображения, принадлежащие объявлениям текущего пользователя, исключая возможность удаления чужих файлов

4.2.5 Класс ItemModerationRequestViewSet

ItemModerationRequestViewSet — класс для управления заявками на модерацию объявлений. Доступен исключительно суперпользователю (IsSuperUser).

Описание методов класса ItemModerationRequestViewSet представлено в таблице ??.

Таблица 4.12 – Описание методов класса ItemModerationRequestViewSet

Название метода	Аргументы метода	Описание метода
1	2	3
get_queryset	—	Возвращает список заявок с предзагрузкой связанных данных (объявление, владелец, категория, изображения). Поддерживает фильтрацию по параметру status
approve	request, pk	Одобрять заявку на модерацию: вызывает сервис approve_item_moderation_request, переводящий объявление в статус available. При статусе заявки, отличном от pending, возвращает HTTP 400
reject	request, pk	Отклоняет заявку: требует непустого поля reason. Вызывает сервис reject_item_moderation_request с указанием причины отказа

4.3 Модуль bookings

Модуль bookings реализует полный жизненный цикл аренды: создание бронирования, оплату через СБП, подтверждение передачи и возврата вещи, управление залогом и отзывами.

4.3.1 Класс BookingViewSet

BookingViewSet — основной класс модуля, реализующий полный цикл управления бронированием. Все методы требуют аутентификации (IsAuthenticated). Поддерживает фильтрацию по параметру role: incoming — входящие бронирования (пользователь является арендодателем), outgoing — исходящие (пользователь является арендатором).

Описание методов класса BookingViewSet представлено в таблице ??.

Таблица 4.13 – Описание методов класса BookingViewSet

Название метода	Аргументы метода	Описание метода
1	2	3
get_queryset	—	Возвращает бронирования, в которых текущий пользователь является арендатором или арендодателем. Поддерживает параметр role для фильтрации по роли
perform_create	serializer	Создаёт бронирование: проверяет, что пользователь не бронирует собственную вещь и что его профиль заполнен. Сохраняет renter как текущего пользователя
change_status	booking, new_status	Вспомогательный метод: устанавливает инициатора смены статуса, вызывает change_status на модели и возвращает сериализованный ответ
ensure_renter	booking	Вспомогательный метод: проверяет, что текущий пользователь является арендатором. Возвращает HTTP 403 при несоответствии
confirm	request, pk	Арендодатель подтверждает бронирование. Переводит статус pending в confirmed. Доступен только владельцу объявления
cancel	request, pk	Отменяет бронирование. Доступен обоим участникам сделки
renter_cancel	request, pk	Отмена бронирования арендатором
complete	request, pk	Арендодатель завершает аренду. Переводит статус в completed. Доступно только при payment_status = paid

Продолжение таблицы ??

1	2	3
start_payment	request, pk	Иницирует оплату через СБП. Генерирует payment_reference (UUID), устанавливает payment_status = pending и срок действия ссылки 15 минут. Возвращает QR-payload. Доступен только арендатору
confirm_payment	request, pk	Подтверждает факт оплаты. Устанавливает payment_status = paid и фиксирует paid_at. Доступен только арендатору
confirm_renter_pickup	request, pk	Арендатор подтверждает получение вещи. Принимает необязательный файл photo
confirm_owner_return	request, pk	Арендодатель подтверждает возврат вещи. Принимает необязательный файл photo
return_deposit	request, pk	Арендодатель инициирует возврат залога арендатору
request_refund	request, pk	Арендатор запрашивает возврат средств
process_refund	request, pk	Администратор обрабатывает запрос на возврат средств
review	request, pk	Арендатор оставляет отзыв о завершённом бронировании. Допускается только один отзыв; повторная попытка возвращает HTTP 400

4.4 Модуль arenda_site

Модуль arenda_site является точкой входа Django-проекта и содержит глобальные настройки, корневую маршрутизацию и конфигурацию ASGI.

Настройки проекта сосредоточены в файле `settings.py`, корневая маршрутизация — в `urls.py`, запуск ASGI-сервера — через `asgi.py`.

4.4.1 `settings.py`

`settings.py` — файл глобальных настроек проекта, определяющий параметры базы данных, аутентификации, установленных приложений, безопасности и инфраструктуры.

Описание параметров модуля `settings.py` представлено в таблице ??.

Таблица 4.14 – Описание параметров модуля `settings.py`

Название параметра	Значение	Описание параметра
1	2	3
DEBUG	False	Производственный режим; отключает вывод отладочной информации
ALLOWED_HOSTS	mokitoki.vti-cloud.ru, localhost, 127.0.0.1, web	Список разрешённых хостов для обработки входящих запросов
INSTALLED_APPS	daphne, users, items, bookings, channels, drf_spectacular, corsheaders, django_filters, rest_framework и стандартные приложения Django	Перечень подключённых приложений проекта
AUTH_USER_MODEL	users.User	Переопределяет стандартную модель пользователя Django на кастомную

Продолжение таблицы ??

1	2	3
DATABASES	PostgreSQL 15, хост db, порт 5432	Параметры подключения к базе данных; значения считываются из переменных окружения файла .env
REST_FRAMEWORK	JWTAuthentication, PageNumber Pagination (10), DjangoFilterBackend, SearchFilter, OrderingFilter	Настройки Django REST Framework: аутентификация, пагинация и фильтрация
SIMPLE_JWT	access — 60 минут, refresh — 1 сутки	Время жизни JWT-токенов
CHANNEL_LAYERS	Redis, хост redis:6379	Настройка слоя каналов Django Channels
CORS_ALLOWED_ORIGINS	http://localhost, http://127.0.0.1, http://mokitoki.vti-cloud.ru	Список разрешённых источников для кросс-доменных запросов
MEDIA_ROOT / MEDIA_URL	директория media/, префикс /media/	Настройки хранения и отдачи медиафайлов
EMAIL_BACKEND	smtp.EmailBackend, Gmail, порт 587, TLS	Настройки отправки электронной почты
SPECTACULAR_SETTINGS	версия 1.0.0, заголовков Diplom API	Настройки генерации OpenAPI-документации; Swagger UI доступен по адресу /api/docs/
SECURE_BROWSER_XSS_FILTER, SECURE_CONTENT_TYPE_NOSNIFF	True	Заголовки безопасности: защита от XSS и MIME-sniffing

Продолжение таблицы ??

1	2	3
LANGUAGE_CODE / TIME_ZONE	ru-ru / Europe/Moscow	Локализация и часовой пояс приложения

4.4.2 urls.py

urls.py — файл корневой маршрутизации проекта, подключающий URL-конфигурации всех приложений и стандартные маршруты аутентификации.

Описание маршрутов модуля urls.py представлено в таблице ??.

Таблица 4.15 – Описание маршрутов модуля urls.py

Маршрут	Подключаемый модуль	Описание
1	2	3
/admin/	django.contrib.admin	Стандартная панель администратора Django
/api/token/	TokenObtainPairView	Получение пары JWT-токенов по логину и паролю
/api/token/refresh/	TokenRefreshView	Обновление access-токена по refresh-токену
/api/users/	users.urls	Маршруты модуля пользователей
/api/items/	items.urls	Маршруты модуля объявлений
/api/bookings/	bookings.urls	Маршруты модуля бронирований
/api/schema/	SpectacularAPIView	Скачивание OpenAPI-схемы в формате YAML
/api/docs/	SpectacularSwaggerView	Swagger UI — интерактивная документация API

4.5 Интеграционное тестирование API

Интеграционное тестирование проводилось с использованием APIClient. Проверялось взаимодействие между модулями users, items и bookings в рамках полного цикла аренды.

Результаты интеграционного тестирования представлены в таблице ??.

Таблица 4.16 – Результаты интеграционного тестирования API

Название теста	Входные данные	Ожидаемый результат
1	2	3
Регистрация нового пользователя	POST /api/users/register/: username='renter1', password='pass123', email='renter@test.com', email_code='123456'	status_code=201, в теле ответа username='renter1'
Получение JWT-токена	POST /api/token/: username='renter1', password='pass123'	status_code=200, в теле ответа 'access', 'refresh'
Обновление профиля физического лица	PUT /api/users/profile/: user_type='individual', full_name='Иван Иванов', passport_series='1234', passport_number='123456', inn='123456789012'	status_code=200, profile_completed=true
Обновление профиля без обязательного поля	PUT /api/users/profile/: user_type='individual', passport_number='123456' (без passport_series)	status_code=400, в теле ответа 'passport_series'

Продолжение таблицы ??

1	2	3
Создание объявления с незаполненным профилем	POST /api/items/: title='Дрель', price_per_day=500 (profile_completed=false)	status_code=403
Создание объявления с заполненным профилем	POST /api/items/: title='Дрель', description='Мощная дрель', price_per_day=500	status_code=201, status='pending', в БД создана заявка ItemModerationRequest
Одобрение объявления администратором	POST /api/items/moderation-requests/id/approve/ (superuser)	status_code=200, status='approved', объявление status='available'
Отклонение объявления администратором	POST /api/items/moderation-requests/id/reject/: reason='Недостаточно фото'	status_code=200, status='rejected', rejection_reason='Недостаточно фото'
Загрузка изображения к объявлению	POST /api/items/upload-image/: item=1, image=test.jpg	status_code=201, item.images.count()=1
Удаление изображения объявления	DELETE /api/items/images/id/	status_code=204, запись удалена из БД
Получение занятых дат объявления	GET /api/items/id/booked_ranges/	status_code=200, в ответе [start_date='2026-05-10', end_date='2026-05-12']

Продолжение таблицы ??

1	2	3
Создание бронирования	POST /api/bookings/: item=1, start_date='2026-07-01', end_date='2026-07-04'	status_code=201, status='pending', total_price=1500.00
Попытка забронировать собственное объявление	POST /api/bookings/: item=1 (renter=owner)	status_code=400
Создание бронирования с пересекающимися датами	POST /api/bookings/: item=1, start_date='2026-07-03', end_date='2026-07-06' (уже занято 01–04)	status_code=400
Редактирование бронирования	PATCH /api/bookings/id/: end_date='2026-07-05'	status_code=405
Подтверждение бронирования владельцем	POST /api/bookings/id/confirm/ (от владельца)	status_code=200, status='confirmed'
Отмена бронирования арендатором	POST /api/bookings/id/cancel/ (от арендатора)	status_code=200, status='cancelled'
Инициация оплаты до подтверждения бронирования	POST /api/bookings/ id/start_payment/ (status='pending')	status_code=400, payment_status='unpaid'
Инициация оплаты СБП	POST /api/bookings/ id/start_payment/ (status='confirmed')	status_code=200, payment_status='pending', в ответе 'qr_payload', provider='sbp_stub'

Продолжение таблицы ??

1	2	3
Подтверждение оплаты	POST /api/bookings/id/confirm_payment/	status_code=200, payment_status='paid', paid_at != null
Завершение аренды до оплаты	POST /api/bookings/id/complete/ (payment_status='unpaid')	status_code=400, status='confirmed'
Завершение аренды после оплаты	POST /api/bookings/id/complete/ (payment_status='paid')	status_code=200, status='completed'
Сохранение цены после изменения объявления	POST /api/bookings/id/start_payment/ после изменения price_per_day с 1500 на 3000	status_code=200, amount=3000.00 (зафиксированный total_price)
Отзыв от арендатора после завершения	POST /api/bookings/id/review/: rating=5, comment='Отличная аренда'	status_code=201, объект Review создан в БД
Повторный отзыв от того же арендатора	POST /api/bookings/id/review/: rating=1, comment='Хочу изменить'	status_code=400, rating первого отзыва сохраняется =5
Отзыв от владельца объявления	POST /api/bookings/id/review/: rating=5 (от owner)	status_code=403, Review не создаётся
Скрытие отзыва модератором	review.hide (reason='Offensive language')	is_hidden=true, reviews_count=0, average_rating=0 у объявления и владельца

ЗАКЛЮЧЕНИЕ

Разработанная серверная часть онлайн-площадки для сдачи вещей в аренду представляет собой полностью функциональный REST API, обеспечивающий работу всех ключевых бизнес-процессов платформы: регистрацию и верификацию пользователей, публикацию объявлений с системой премодерации, управление бронированиями и оплатой через Систему быстрых платежей.

В заключении можно выделить следующие основные результаты работы:

1. В ходе анализа предметной области были рассмотрены принципы функционирования онлайн-площадок для аренды вещей, изучены существующие решения. Были выявлены их достоинства и недостатки, определены ключевые функциональные требования к проектируемой системе и обоснована необходимость разработки серверной части с учётом специфики российского рынка.

2. Спроектирована база данных серверной части приложения. Выделены основные сущности: пользователи, объявления о сдаче вещей в аренду, бронирования с отслеживанием платёжного статуса и статуса залога, заявки на модерацию объявлений, отзывы. Установлены связи между сущностями, определены составные индексы для оптимизации запросов.

3. Разработан REST API веб-приложения. Реализован полный жизненный цикл бронирования: от создания заявки и её подтверждения владельцем до оплаты через Систему быстрых платежей, подтверждения передачи и возврата вещи с фотофиксацией, а также возврата залога. Реализована система модерации объявлений с сохранением снимков данных на момент подачи заявки. Для обеспечения безопасности применена JWT-аутентификация.

4. Выполнена интеграция с внешним сервисом DaData для автозаполнения реквизитов организаций и индивидуальных предпринимателей по номеру ИНН. Реализована валидация ИНН, КПП и ОГРНИП по алгоритму кон-

трольных цифр, что исключает ввод некорректных юридических данных при регистрации пользователей.

5. Проведено тестирование разработанного API. Реализованы модульные тесты моделей, а также системные тесты сквозных сценариев с использованием APIClient.

Все требования, объявленные в техническом задании, были полностью реализованы, все задачи, поставленные в начале разработки проекта, были также решены.

Разработанная серверная часть представляет собой готовое к развёртыванию решение и может быть использована как основа для создания коммерческой платформы аренды или масштабирована путём добавления новых категорий товаров и интеграции с реальными платёжными шлюзами.

Перспективой дальнейшего развития является интеграция с реальным провайдером СБП, интеграция с реальным криптопровайдером для ЭЦП, а также добавление курьерской доставки в веб-приложении.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Российская Федерация. Законы. О защите прав потребителей: Закон РФ от 7 февраля 1992г N 2300-1 – Текст : электронный // КонсультантПлюс: справочно-правовая система. URL:https://www.consultant.ru/document/cons_doc_LAW_305/ (дата обращения: 04.05.2026).

2. Российская Федерация. Законы. Глава 34 «Аренда». Гражданский кодекс РФ – Текст : электронный // КонсультантПлюс: справочно-правовая система. — URL: https://www.consultant.ru/document/cons_doc_LAW_9027/cac65a05697011caaa0eff7d476d867137f1f92a/ (дата обращения: 04.05.2026).

3. Российская Федерация. Законы. О проведении эксперимента по установлению специального налогового режима ”Налог на профессиональный доход”: Федеральный закон от 27.11.2018 № 422-ФЗ – Текст : электронный // КонсультантПлюс: справочно-правовая система. URL:https://www.consultant.ru/document/cons_doc_LAW_311977/ (дата обращения: 04.05.2026).

4. Российская Федерация. Законы. О персональных данных: Федеральный закон от 27.07.2006 № 152-ФЗ – Текст : электронный // КонсультантПлюс: справочно-правовая система. URL:https://www.consultant.ru/document/cons_doc_LAW_61801/ (дата обращения: 05.05.2026).

5. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем.— Москва : Стандартинформ, 2010. — Текст : непосредственный.

6. ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы.— Москва : Стандартинформ, 2020. — Текст : непосредственный.

7. ГОСТ 34.201-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Виды, комплектность и обозначение документов при создании автоматизированных систем. – Москва : Стандарт информ, 2020. – Текст : непосредственный.

8. Дронов, В. А. Django 4. Практика создания веб-сайтов на Python. — СПб.: БХВ-Петербург, 2023. — 800 с. — ISBN 978-5-9775-1774-4. — Текст : непосредственный.

9. Агафонов, А. А. Основы технологий баз данных: учебное пособие / А.А. Агафонов, А.М. Белов. – Самара: Издательство Самарского университета, 2023. – 304 с. – ISBN 978-5-7883-1915-5. – Текст : непосредственный.

10. Буч, Г. Язык UML. Руководство пользователя / Г. Буч, Д. Рамбо, А. Якобсон. – Москва : ДМК Пресс, 2006. – 496 с. – ISBN 5-94074-334-X. – Текст : непосредственный.

11. Новиков, Ф. А. Учебно-методическое пособие по дисциплине «Анализ и проектирование на UML»: учебно-методическое пособие / Ф.А. Новиков. — Санкт-Петербург : НИУ ИТМО, 2007. — 286 с. — Текст : электронный.

12. Арно, Л. Проектирование веб-API / Пер. с англ. Д. А. Беликова. – М.: ДМК Пресс, 2020. – 440 с. – ISBN 978-5-97060-861-6. – Текст : непосредственный.

13. Бхаргава, А. Грокаем алгоритмы. Иллюстрированное пособие для программистов и любопытствующих. - СПб.: Питер, 2017. - 288 с. – ISBN 978-5-496-02541-6. – Текст : непосредственный.

14. Милл, И. Docker на практике / пер. с англ. Д.А. Беликов. – М.: ДМК Пресс, 2020. – 516 с. – ISBN 978-5-97060-772-5. – Текст : непосредственный.

15. Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. — 2-е изд., перераб. и доп. — СПб.: БХВ-Петербург, 2019. — 608 с. – ISBN 978-5-9775-3994-4. – Текст : непосредственный.

16. Мэтиз, Э. Изучаем Python: программирование игр, визуализация данных, веб-приложения. 3-е изд. — СПб.: Питер, 2020 — 512 с. — ISBN 978-5-4461-1528-0. — Текст : непосредственный.

17. Чаплыгин, А.А. Основы дипломного проектирования: учебное пособие для студентов направления подготовки 09.03.04 Программная инженерия, направленность (профиль) «Разработка программноинформационных систем» / А.А. Чаплыгин, В.В. Апальков, А.В. Малышев ; Юго-Зап. гос. ун-т. — Курск, 2022. — 102 с. — Библиогр.: 76–79 с. — ISBN 978-5-907407-65-7. — Текст : непосредственный.

18. Амундсен, М. RESTful Web API. Паттерны и практики. — Астана: «Спринт Бук», 2025. — 464 с. — ISBN 978-601-08-4867-2. — Текст : непосредственный.

19. Свейгарт, Э. Рекурсивная книга о рекурсии. — СПб.: Питер, 2023. — 336 с. — ISBN 978-5-4461-2393-3. — Текст : непосредственный.

20. Аттарди, Д. Web API. Сборник рецептов: пер. с англ. - Астана: АЛИСТ, 2025. 304 с. — ISBN 978-601-12-3681-2. - Текст : непосредственный.

21. Константинов, С. С. API как искусство: разработка, поддержка, интеграция. - СПб.: БХВ-Петербург, 2025. - 304 с. — ISBN 978-5-9775-2084-3. — Текст : непосредственный.

22. Светлаков В. И. Архитектура бэкенда. API для надежных корпоративных приложений. СПб.: БХВ-Петербург, 2026. - 256 с. — ISBN 978-5-9775-2102-4. — Текст : непосредственный.

23. Нисчал Н. Python — это просто. Пошаговое руководство по программированию и анализу данных: Пер. с англ. — СПб.: БХВ-Петербург, 2023. — 416 с. — ISBN 978-5-9775-6849-4. — Текст : непосредственный.

24. Левашов, П. Python с нуля. — СПб.: Питер, 2024. — 448 с. — ISBN 978-5-4461-2145-8. — Текст : непосредственный

25. Афанасьев, А. Ю. Веб-программирование : учебник и практикум для академического бакалавриата / А. Ю. Афанасьев. — 2-е изд., перераб. и доп. — Москва: Юрайт, 2019. — 333 с. — ISBN 978-5-534-01432-5. — Текст : непосредственный.

26. Кальб, И. Объектно-ориентированное программирование с помощью Python / Ирв Кальб ; [перевод с английского М. А. Райтмана]. — Москва : Эксмо, 2024. — 512 с. — ISBN 978-5-04-186627-3. — Текст : непосредственный.

27. Юн, Ц. Рецепты Python. Коллекция лучших техник программирования. — СПб.: Питер, 2024. — 512 с. — ISBN 978-5-4461-2156-4. — Текст : непосредственный.

28. Моргунов, Е. П. PostgreSQL. Основы языка SQL/ Е. П. Моргунов. — Санкт-Петербург : БХВ-Петербург, 2018. — 336 с. — ISBN-978-5-9775-3697. — Текст : непосредственный.

29. Соммервиль, И. Инженерия программного обеспечения /И. Соммервиль ; пер. с англ. — 10-е изд. — Москва : Вильямс, 2019. — 1072 с. — ISBN-978-5-907036-78-5. - Текст : непосредственный.

30. Атчисон, Л. Масштабирование приложений. Выращивание сложных систем. - СПб.: Питер, 2018. - 256 с. — ISBN 978-5-496-02952-0 — Текст : непосредственный.

31. Дронов В. А. Фишерман Л. В. Git. Практическое руководство. Управление и контроль версий в разработке программного обеспечения - СПб.: Наука и техника, 2021. - 304 с. — ISBN 978-5-94387-547-2 — Текст : непосредственный.

32. Кольцов, Д. В. Python: Создаем программы и игры. 2-е издание. - СПб.: Наука и Техника, 2019.-400 с. — ISBN 978-5-94387-778-0. — Текст : непосредственный.

33. Вигерс, К. Разработка требований к программному обеспечению: [практические приёмы сбора требований и управления ими при разработке программных продуктов : пер. с англ.] / К. Вигерс, Д. Битти. — 3-е изд., доп. — Санкт-Петербург : BHV, 2020. — 736 с. — ISBN 978-5-9775-3348-5. — Текст : непосредственный.

34. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин ; пер. с англ. А. Кисилева. — Санкт-

Петербург : Питер, 2018. – 351 с. – ISBN 978-5-4461-0772-8. — Текст : непосредственный.

35. Белик, А. Г. Проектирование и архитектура программных систем : учебное пособие / А. Г. Белик, В. Н. Цыганенко. – Омск : ОмГТУ, 2016. – 96 с. — ISBN 978-5-8149-2258-8. — Текст : непосредственный.

36. Дерек де Йонге. NGINX. Книга рецептов. / пер. с англ. Д. А. Беликова. – М.: ДМК Пресс, 2020. – 176 с. – ISBN 978-5-97060-790-9. – Текст : непосредственный.

37. Ткачев, О. А. PostgreSQL: SQL + PL/pgSQL для тех, кто хочет стать профессионалом — СПб.: Издательство Наука и Техника, 2024. — 480 с. — ISBN 978-5-907592-32-2. — Текст : непосредственный.

38. Лейси, Н. Python, например. — СПб.: Питер, 2021. — 208 с. — ISBN 978-5-4461-1826-7. — Текст : непосредственный.

39. Вальдуриес, П. Принципы организации распределенных баз данных / пер. с англ. А. А. Слинкина. – М.: ДМК Пресс, 2021. – 672 с. – ISBN 978-5-97060-391-8 — Текст : непосредственный.

40. Пьюривал, С. Основы разработки веб-приложений. — СПб.: Питер, 2015. — 272 с. – ISBN 978-5-496-01226-3. – Текст : непосредственный.

41. PostgreSQL Documentation : официальный сайт. – PostgreSQL, 1996 – URL: <https://www.postgresql.org/docs/> (дата обращения: 07.05.2026).

42. Django Documentation : официальный сайт. – Django Software Foundation, 2005 – URL: <https://docs.djangoproject.com/en/6.0/> (дата обращения: 04.05.2026).

43. Python Documentation : официальный сайт. – Python Software Foundation, 2001 – URL: <https://www.python.org/doc/> (дата обращения: 11.05.2026).

44. Git Documentation : официальный сайт. – Software Freedom Conservancy, 2005 – URL: <https://git-scm.com/docs/git> (дата обращения: 11.05.2026).

45. Docker Documentation : официальный сайт. – Docker Inc., 2013 – URL: <https://docs.docker.com> (дата обращения: 12.05.2026).

46. Visual Studio Code Documentation : официальный сайт. – Microsoft, 2015 – URL: <https://code.visualstudio.com/docs> (дата обращения: 05.05.2026).
47. Nginx Documentation : официальный сайт. – F5, Inc., 2004 – URL: <https://nginx.org/ru/docs/> (дата обращения: 14.05.2026).
48. Gunicorn Documentation : официальный сайт. – Gunicorn, 2010 – URL: <https://gunicorn.org/deploy/> (дата обращения: 14.05.2026).
49. Swagger Documentation : официальный сайт. – SmartBear Software, 2011 – URL: <https://docs.swagger.io/сpec.html> (дата обращения: 14.05.2026).
50. HTTP Documentation : официальный сайт. – HTTP Working Group, 1999 – URL: <https://httpwg.org/specs/> (дата обращения: 14.05.2026).

ПРИЛОЖЕНИЕ А

Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.0.

ВКРБ 2206116.09.03.04.26.006				1									
Сведения о ВКРБ Минобрнауки России Юго-Западный государственный университет Кафедра программной инженерии ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА ПО ПРОГРАММЕ БАКАЛАВРИАТА «Бизнес-проект «Онлайн-площадка для сдачи вещей в аренду». Разработка серверной части» Руководитель ВКРБ к.т.н, доцент Петрик Елена Анатольевна Автор ВКРБ студент группы ПО-216 Ермаков Владислав Артемович													
				ВКРБ 2206116.09.03.04.26.006									
				<table border="1"> <tr> <td>Лит.</td> <td>Место</td> <td>Датум</td> </tr> <tr> <td></td> <td></td> <td></td> </tr> </table>				Лит.	Место	Датум			
Лит.	Место	Датум											
				<table border="1"> <tr> <td>Сведения о ВКРБ</td> <td>Лист 1</td> <td>Из листов 10</td> </tr> <tr> <td></td> <td></td> <td></td> </tr> </table>				Сведения о ВКРБ	Лист 1	Из листов 10			
Сведения о ВКРБ	Лист 1	Из листов 10											
				<table border="1"> <tr> <td>Выпускная квалификационная работа бакалавра</td> <td>ЮЗГУ ПО-216</td> </tr> <tr> <td></td> <td></td> </tr> </table>				Выпускная квалификационная работа бакалавра	ЮЗГУ ПО-216				
Выпускная квалификационная работа бакалавра	ЮЗГУ ПО-216												

Рисунок А.1 – Сведения о ВКРБ

Цель и задачи разработки

Целью выпускной квалификационной работы является проектирование, разработка и тестирование серверной части веб-приложения, обеспечивающей хранение данных, бизнес-логику и предоставление REST API для онлайн-площадки аренды вещей. Серверная часть предоставляет арендодателям возможность публиковать объявления, а арендаторам — находить, бронировать и оплачивать аренду с гарантиями безопасности сделки. Задачами данной разработки являются:

- изучение и анализ предметной области;
- проектирование архитектуры серверной части и базы данных;
- разработка REST API веб-приложения;
- интеграция с сервисом DaData;
- тестирование разработанного API.

ВКРБ 2206116.09.03.04.26.006			
Фамилия И. О.	Фамилия И. О.	Дата	Лит. Мисс. Значим.
Автор работы	Ермаков В. А.		
Руководитель	Пестрик Е. А.		
Нормоконтроль	Чалыгин А. А.		
Цель и задачи разработки			Лист 2 из 10
Выпускная квалификационная работа бакалавра			ЮЗГУ ПО-216

Рисунок А.2 – Цель и задачи разработки

900 9С 04 03 04 26 006

3

Сравнение существующих онлайн-площадок
для аренды вещей

Параметр	Rentomania	Авито	Арендую.ру	Проектируемая онлайн площадка
Размещение объявлений арендодателем	+	+	+	+
Поиск и фильтрация товаров	+	+	+	+
Онлайн- бронирование	+	—	—	+
Встроенная онлайн- оплата	+	+	—	+
Система рейтинга и отзывов	+	+	—	+
Чат между арендатором и арендодателем	+	+	—	+
Адаптивная вёрстка для мобильных устройств	+	+	—	+
Верификация пользователей	+	—	—	+
Панель аналитики для арендодателя	—	—	—	+
Понятный интерфейс	—	+	—	+
Уведомления о статусе аренды	+	—	—	+

ВКРБ 2206116.09.03.04.26.006			Лит.	Место	Датум
Автор работы	Фамилия И. О.	Имя	Дата	Сравнение существующих онлайн-площадок для аренды вещей	
Руководитель	Ермаков В. А.			Лист 3	Листов 10
Нормоконтроль	Пестрик Е. А.			Выпускная квалификационная работа бакалавра	
	Чалыгин А. А.			ЮЗГУ ПО-216	

Рисунок А.3 – Сравнение существующих онлайн-площадок для аренды вещей



Рисунок А.4 – Компоненты онлайн-площадки для аренды вещей

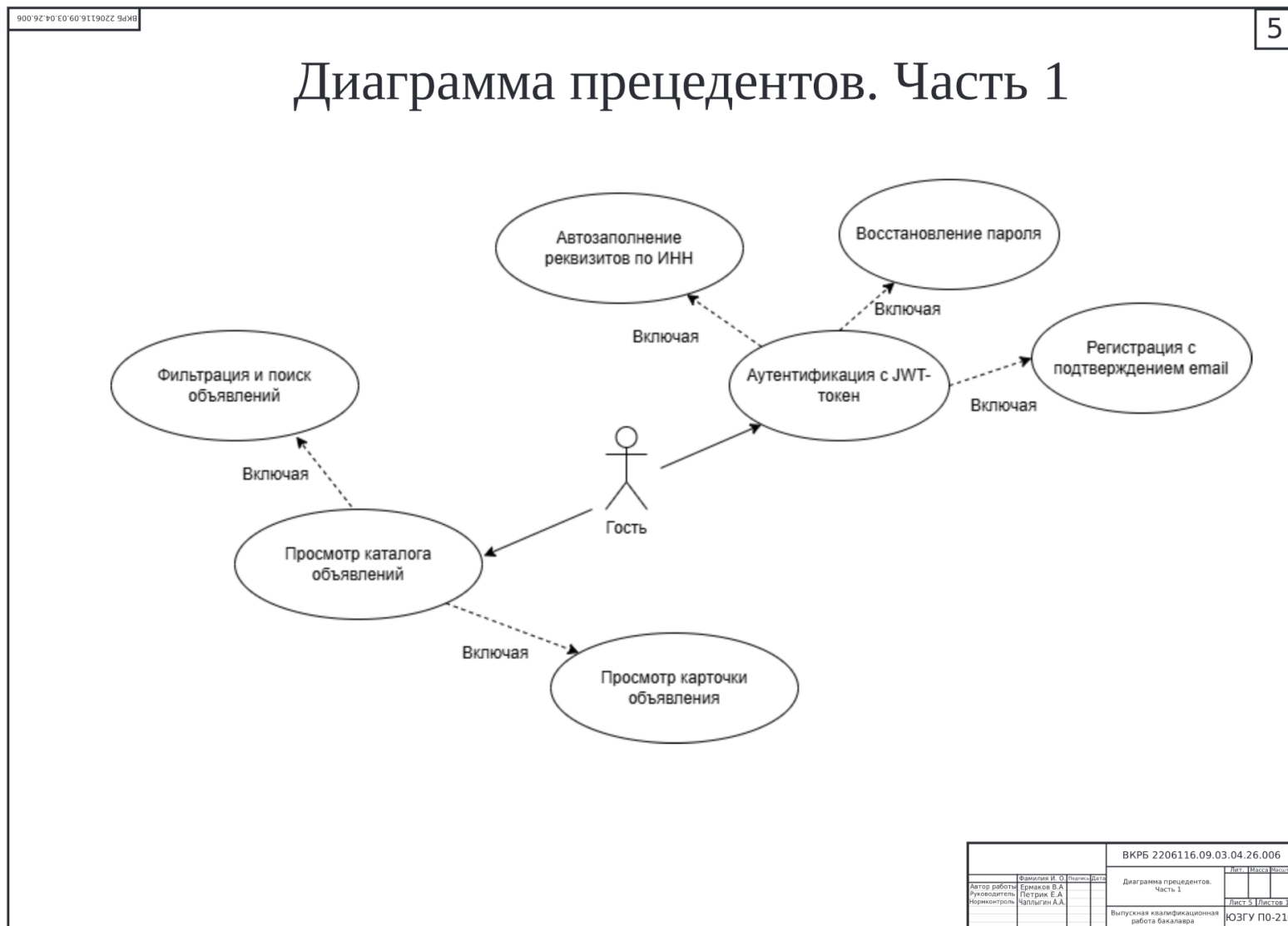


Рисунок А.5 – Диаграмма прецедентов. Часть 1



Рисунок А.6 – Диаграмма прецедентов. Часть 2



Рисунок А.7 – Диаграмма прецедентов. Часть 3



Рисунок А.8 – Диаграмма компонентов

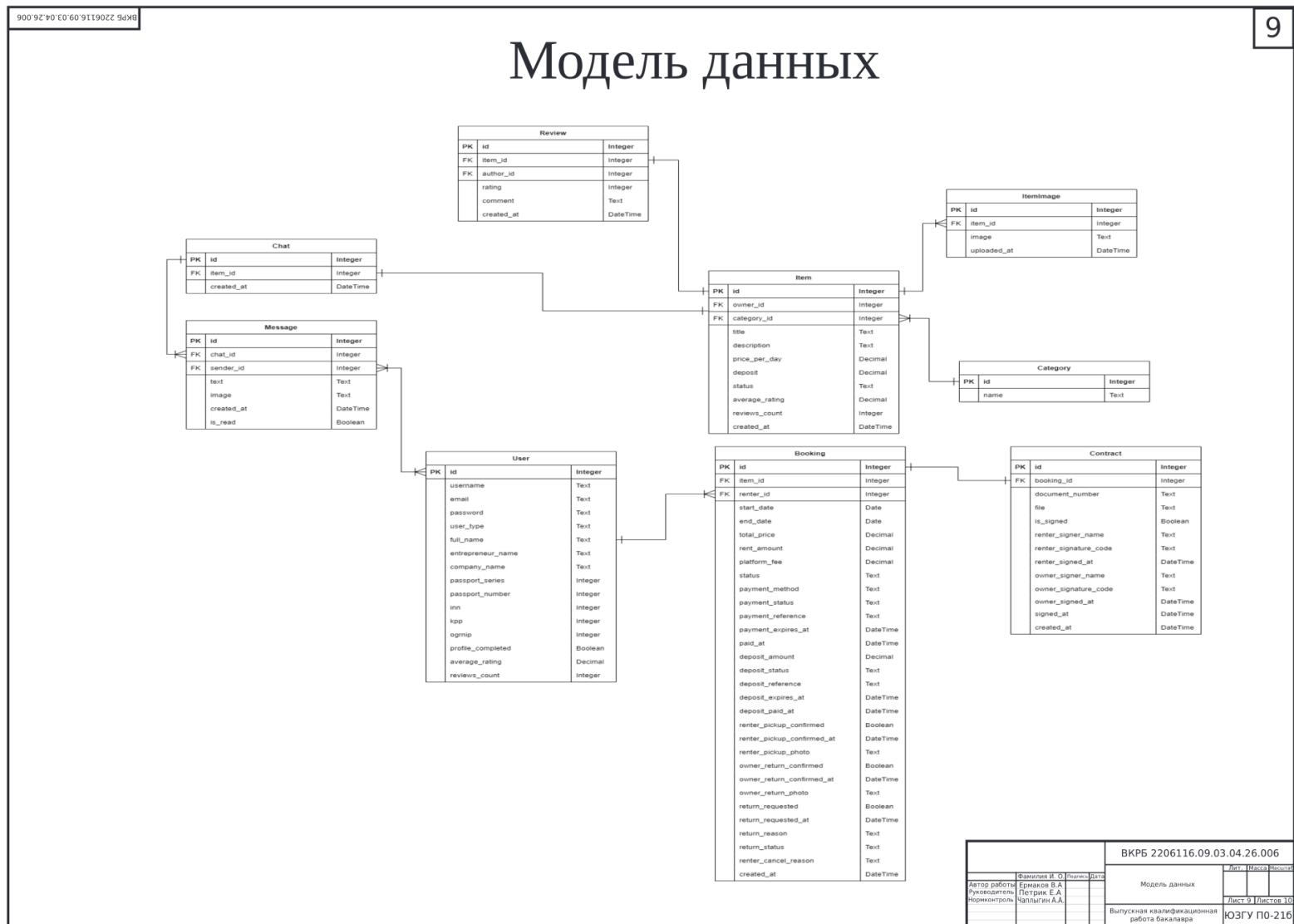


Рисунок А.9 – Модель данных

Заключение

В заключении можно выделить следующие основные результаты работы:

1. В ходе анализа предметной области были рассмотрены принципы функционирования онлайн-площадок для аренды вещей, изучены существующие решения. Были выявлены их достоинства и недостатки, определены ключевые функциональные требования к проектируемой системе и обоснована необходимость разработки серверной части с учётом специфики российского рынка.
2. Спроектирована база данных серверной части приложения. Выделены основные сущности: пользователи, объявления о сдаче вещей в аренду, бронирования с отслеживанием платёжного статуса и статуса залога, заявки на модерацию объявлений, отзывы. Установлены связи между сущностями, определены составные индексы для оптимизации запросов.
3. Разработан REST API веб-приложения. Реализован полный жизненный цикл бронирования: от создания заявки и её подтверждения владельцем до оплаты через Систему быстрых платежей, подтверждения передачи и возврата вещи с фотофиксацией, а также возврата залога. Реализована система модерации объявлений с сохранением снимков данных на момент подачи заявки. Для обеспечения безопасности применена JWT-аутентификация.
4. Выполнена интеграция с внешним сервисом DaData для автозаполнения реквизитов организаций и индивидуальных предпринимателей по номеру ИНН. Реализована валидация ИНН, КПП и ОГРНИП по алгоритму контрольных цифр, что исключает ввод некорректных юридических данных при регистрации пользователей.
5. Проведено тестирование разработанного API. Реализованы модульные тесты моделей, а также системные тесты сквозных сценариев с использованием APIClient.

ВКРБ 2206116.09.03.04.26.006			
Фамилия И. О.	Имя	Отчество	Дата
Автор работы	Ермаков В. А.		
Руководитель	Пестрик Е. А.		
Нормоконтроль	Чалыгин А. А.		
Заключение			Лист 10
Выпускная квалификационная работа бакалавра			Листов 10
			ЮЗГУ ПО-216

Рисунок А.10 – Заключение

ПРИЛОЖЕНИЕ Б

Фрагменты исходного кода программы

models.py

```
1 from django.db import models
2 from django.conf import settings
3 from django.core.validators import MaxValueValidator, MinValueValidator
4 from django.db.models import Avg
5
6 class Category(models.Model):
7     name = models.CharField(max_length=100)
8
9     def __str__(self):
10         return self.name
11 class Item(models.Model):
12
13     STATUS_CHOICES = (
14         ('pending', 'На модерации'),
15         ('available', 'Опубликовано'),
16         ('unavailable', 'Недоступно'),
17         ('blocked', 'Заблокировано'),
18     )
19
20     owner = models.ForeignKey(
21         settings.AUTH_USER_MODEL,
22         on_delete=models.CASCADE,
23         related_name='items'
24     )
25
26     title = models.CharField(max_length=255)
27     description = models.TextField()
28     price_per_day = models.DecimalField(max_digits=10, decimal_places=2)
29     deposit = models.DecimalField(
30         max_digits=10,
31         decimal_places=2,
32         default=0,
33         verbose_name='Залог'
34     )
35     average_rating = models.DecimalField(
36         max_digits=3,
37         decimal_places=2,
38         default=0
39     )
40
41     reviews_count = models.IntegerField(default=0)
42     status = models.CharField(
43         max_length=20,
44         choices=STATUS_CHOICES,
45         default='available'
46     )
47
48     created_at = models.DateTimeField(auto_now_add=True)
49
```

```

50     category = models.ForeignKey(
51         Category,
52         on_delete=models.SET_NULL,
53         null=True,
54         blank=True,
55         related_name='items'
56     )
57     def __str__(self):
58         return self.title
59
60
61 class ItemImage(models.Model):
62     item = models.ForeignKey(
63         Item,
64         on_delete=models.CASCADE,
65         related_name='images'
66     )
67     image = models.ImageField(upload_to='items/')
68     uploaded_at = models.DateTimeField(auto_now_add=True)
69
70     def __str__(self):
71         return f"Image for {self.item.title}"
72
73     class Meta:
74         ordering = ['-uploaded_at']
75
76
77 class ItemModerationRequest(models.Model):
78     ACTION_CREATE = 'create'
79     ACTION_UPDATE = 'update'
80     STATUS_PENDING = 'pending'
81     STATUS_APPROVED = 'approved'
82     STATUS_REJECTED = 'rejected'
83
84     ACTION_CHOICES = (
85         (ACTION_CREATE, 'Создание'),
86         (ACTION_UPDATE, 'Редактирование'),
87     )
88     STATUS_CHOICES = (
89         (STATUS_PENDING, 'Ожидает рассмотрения'),
90         (STATUS_APPROVED, 'Одобрена'),
91         (STATUS_REJECTED, 'Отклонена'),
92     )
93
94     item = models.ForeignKey(
95         Item,
96         on_delete=models.SET_NULL,
97         null=True,
98         blank=True,
99         related_name='moderation_requests',
100     )
101     submitted_by = models.ForeignKey(
102         settings.AUTH_USER_MODEL,
103         on_delete=models.CASCADE,

```

```

104         related_name='item_moderation_requests',
105     )
106     action = models.CharField(max_length=20, choices=ACTION_CHOICES)
107     status = models.CharField(max_length=20, choices=STATUS_CHOICES, default=
STATUS_PENDING)
108     item_snapshot = models.JSONField(default=dict, blank=True)
109     user_snapshot = models.JSONField(default=dict, blank=True)
110     rejection_reason = models.TextField(blank=True)
111     reviewed_by = models.ForeignKey(
112         settings.AUTH_USER_MODEL,
113         on_delete=models.SET_NULL,
114         null=True,
115         blank=True,
116         related_name='reviewed_item_moderation_requests',
117     )
118     reviewed_at = models.DateTimeField(null=True, blank=True)
119     created_at = models.DateTimeField(auto_now_add=True)
120     updated_at = models.DateTimeField(auto_now=True)
121
122     class Meta:
123         ordering = ['-created_at']
124         indexes = [
125             models.Index(fields=['status', '-created_at']),
126             models.Index(fields=['submitted_by', '-created_at']),
127             models.Index(fields=['item', 'status']),
128         ]
129
130     def __str__(self):
131         return f'{self.get_action_display()} объявления {self.item}'
132
133
134     class ItemVideo(models.Model):
135         item = models.ForeignKey(
136             Item,
137             on_delete=models.CASCADE,
138             related_name='videos'
139         )
140         video = models.FileField(upload_to='items/videos/')
141         uploaded_at = models.DateTimeField(auto_now_add=True)
142
143     def __str__(self):
144         return f"Video for {self.item.title}"
145
146     class Meta:
147         ordering = ['-uploaded_at']
148
149
150     class ItemReview(models.Model):
151         item = models.ForeignKey(
152             Item,
153             on_delete=models.CASCADE,
154             related_name='reviews',
155         )
156         author = models.ForeignKey(

```

```

157         settings.AUTH_USER_MODEL,
158         on_delete=models.CASCADE,
159         related_name='item_reviews',
160     )
161     rating = models.IntegerField(
162         validators=[MinValueValidator(0), MaxValueValidator(5)],
163     )
164     comment = models.TextField(blank=True)
165     created_at = models.DateTimeField(auto_now_add=True)
166
167     class Meta:
168         ordering = ['-created_at']
169         indexes = [
170             models.Index(fields=['item', '-created_at']),
171             models.Index(fields=['author', '-created_at']),
172         ]
173
174     def save(self, *args, **kwargs):
175         is_new = self.pk is None
176         super().save(*args, **kwargs)
177         if is_new:
178             self.update_item_rating_counters(item=self.item)
179
180     def delete(self, *args, **kwargs):
181         item = self.item
182         super().delete(*args, **kwargs)
183         self.update_item_rating_counters(item=item)
184
185     @staticmethod
186     def update_item_rating_counters(*, item=None):
187         if item is None:
188             return
189         qs = ItemReview.objects.filter(item=item)
190         item.average_rating = qs.aggregate(avg=Avg('rating'))['avg'] or 0
191         item.reviews_count = qs.count()
192         item.save(update_fields=['average_rating', 'reviews_count'])

```

views.py

```

1 from rest_framework import viewsets
2 from rest_framework.permissions import IsAdminUser, IsAuthenticatedOrReadOnly
3 , IsAuthenticated
4 from users.permissions import IsProfileCompleted
5 from .models import Item, ItemModerationRequest, ItemReview
6 from .permissions import IsOwner
7 from .serializers import ItemModerationRequestSerializer, ItemSerializer,
8 ItemReviewSerializer
9 from rest_framework.decorators import action
10 from django_filters.rest_framework import DjangoFilterBackend
11 from rest_framework import filters
12 from django.db import models
13 from django_filters.rest_framework import DjangoFilterBackend
14 from rest_framework import filters, generics, viewsets
15 from rest_framework.decorators import action
16 from rest_framework.exceptions import PermissionDenied, ValidationError

```

```

15 from rest_framework.parsers import FormParser, MultiPartParser
16 from rest_framework.permissions import IsAuthenticated,
    IsAuthenticatedOrReadOnly
17 from rest_framework.response import Response
18 from rest_framework.viewsets import ModelViewSet
19
20 from users.permissions import IsProfileCompleted
21
22 from .models import Category, Item, ItemImage
23 from .permissions import IsOwner
24 from .serializers import CategorySerializer, ItemImageSerializer,
    ItemSerializer
25 from .services import (
26     approve_item_moderation_request,
27     create_item_moderation_request,
28     reject_item_moderation_request,
29 )
30
31 class CategoryViewSet(ModelViewSet):
32     queryset = Category.objects.order_by('name')
33     serializer_class = CategorySerializer
34
35
36 class IsSuperUser(IsAdminUser):
37     def has_permission(self, request, view):
38         return bool(request.user and request.user.is_authenticated and
39                     request.user.is_superuser)
40
41 class ItemModerationRequestViewSet(viewsets.ReadOnlyModelViewSet):
42     serializer_class = ItemModerationRequestSerializer
43     permission_classes = [IsSuperUser]
44
45     def get_queryset(self):
46         queryset = ItemModerationRequest.objects.select_related(
47             'item',
48             'item__owner',
49             'item__category',
50             'submitted_by',
51             'reviewed_by',
52         ).prefetch_related('item__images', 'item__videos')
53
54         status = self.request.query_params.get('status')
55         if status:
56             queryset = queryset.filter(status=status)
57
58         return queryset
59
60 @action(detail=True, methods=['post'])
61 def approve(self, request, pk=None):
62     moderation_request = self.get_object()
63     if moderation_request.status != ItemModerationRequest.STATUS_PENDING:
64         raise ValidationError({'status': ['Заявка уже рассмотрена.']})
65

```

```

66         approve_item_moderation_request(moderation_request, request.user)
67         return Response(self.get_serializer(moderation_request).data)
68
69     @action(detail=True, methods=['post'])
70     def reject(self, request, pk=None):
71         moderation_request = self.get_object()
72         if moderation_request.status != ItemModerationRequest.STATUS_PENDING:
73             raise ValidationError({'status': ['Заявка уже рассмотрена.']})
74
75         reason = (request.data.get('reason') or '').strip()
76         if not reason:
77             raise ValidationError({'reason': ['Укажите причину отказа.']})
78
79         serialized = self.get_serializer(moderation_request).data
80         reject_item_moderation_request(moderation_request, request.user,
81                                         reason)
82         serialized['status'] = ItemModerationRequest.STATUS_REJECTED
83         serialized['rejection_reason'] = reason
84         return Response(serialized)
85
86     class ItemViewSet(viewsets.ModelViewSet):
87
88         serializer_class = ItemSerializer
89
90         def get_queryset(self):
91
92             user = self.request.user
93
94             if user.is_authenticated:
95                 queryset = Item.objects.filter(
96                     models.Q(status='available') | models.Q(owner=user)
97                 )
98             else:
99                 queryset = Item.objects.filter(status='available')
100
101             min_rating = self.request.query_params.get('min_rating')
102             mine = self.request.query_params.get('mine')
103
104             if mine in ['1', 'true', 'True'] and user.is_authenticated:
105                 queryset = queryset.filter(owner=user)
106
107             if min_rating:
108                 queryset = queryset.filter(average_rating__gte=min_rating)
109
110             return queryset.select_related('owner', 'category').distinct()
111             filter_backends = [DjangoFilterBackend, filters.SearchFilter, filters.
112                               OrderingFilter]
113
114             filterset_fields = ['status', 'category']
115             search_fields = ['title', 'description']
116             ordering_fields = ['price_per_day', 'created_at', 'average_rating']
117             ordering = ['-created_at']

```

```

118 def perform_create(self, serializer):
119     item = serializer.save(owner=self.request.user, status='pending')
120     create_item_moderation_request(
121         item=item,
122         submitted_by=self.request.user,
123         action=ItemModerationRequest.ACTION_CREATE,
124     )
125
126 def perform_update(self, serializer):
127     item = serializer.save(owner=self.request.user, status='pending')
128     create_item_moderation_request(
129         item=item,
130         submitted_by=self.request.user,
131         action=ItemModerationRequest.ACTION_UPDATE,
132     )
133
134 def get_permissions(self):
135     if self.action in ['update', 'partial_update', 'destroy']:
136         return [IsOwner(), IsProfileCompleted()]
137     if self.action == 'create':
138         return [IsAuthenticatedOrReadOnly(), IsProfileCompleted()]
139     return []
140
141 @action(detail=True, methods=['get'])
142 def booked_ranges(self, request, pk=None):
143     item = self.get_object()
144     active_bookings = item.bookings.filter(status__in=['pending', '
145         confirmed']).order_by('start_date')
146     return Response(
147         [
148             {
149                 'start_date': booking.start_date.isoformat(),
150                 'end_date': booking.end_date.isoformat(),
151             }
152             for booking in active_bookings
153         ]
154     )
155
156 @action(detail=True, methods=['get', 'post'], permission_classes=[
157     IsAuthenticatedOrReadOnly])
158 def reviews(self, request, pk=None):
159     item = self.get_object()
160
161     if request.method.lower() == 'get':
162         qs = ItemReview.objects.filter(item=item).select_related('author')
163         .order_by('-created_at')
164         page = self.paginate_queryset(qs)
165         serializer = ItemReviewSerializer(page if page is not None else
166             qs, many=True)
167         if page is not None:
168             return self.get_paginated_response(serializer.data)
169         return Response(serializer.data)
170
171 # POST

```

```

168         if not request.user.is_authenticated:
169             raise PermissionDenied("Войдите в систему.")
170         if item.owner_id == request.user.id:
171             raise PermissionDenied("Нельзя оставить отзыв на своё объявление.")
172
173         serializer = ItemReviewSerializer(data=request.data)
174         serializer.is_valid(raise_exception=True)
175         review = ItemReview.objects.create(
176             item=item,
177             author=request.user,
178             rating=serializer.validated_data['rating'],
179             comment=serializer.validated_data.get('comment', ''),
180         )
181         return Response(ItemReviewSerializer(review).data, status=201)
182
183     class ItemImageUploadView(generics.CreateAPIView):
184         serializer_class = ItemImageSerializer
185         parser_classes = (MultiPartParser, FormParser)
186         permission_classes = [IsAuthenticated]
187
188     def perform_create(self, serializer):
189         item_id = self.request.data.get('item')
190         if not item_id:
191             raise ValidationError({'item': ['Укажите объявление.']})
192
193         try:
194             item = Item.objects.get(id=item_id)
195         except Item.DoesNotExist:
196             raise ValidationError({'item': ['Объявление не найдено.']})
197
198         if item.owner != self.request.user:
199             raise PermissionDenied("Это действие доступно только владельцу объявления.")
200
201         serializer.save(item=item)
202
203
204     class ItemImageDeleteView(generics.DestroyAPIView):
205         serializer_class = ItemImageSerializer
206         permission_classes = [IsAuthenticated]
207
208     def get_queryset(self):
209         return ItemImage.objects.filter(item__owner=self.request.user)

```

serializers.py

```

1 from decimal import Decimal, ROUND_HALF_UP
2 from rest_framework import serializers
3
4 from .models import Category, Item, ItemImage, ItemModerationRequest,
   ItemReview, ItemVideo
5
6 MARKUP_RATE = Decimal('0.20') # 20% наценка платформы
7

```

```

8
9 class ItemImageSerializer(serializers.ModelSerializer):
10     class Meta:
11         model = ItemImage
12         fields = ['id', 'image']
13
14
15 class ItemVideoSerializer(serializers.ModelSerializer):
16     class Meta:
17         model = ItemVideo
18         fields = ['id', 'video']
19
20
21 class CategorySerializer(serializers.ModelSerializer):
22     class Meta:
23         model = Category
24         fields = ['id', 'name']
25
26 class ItemSerializer(serializers.ModelSerializer):
27     images = ItemImageSerializer(many=True, read_only=True)
28     videos = ItemVideoSerializer(many=True, read_only=True)
29     booked_ranges = serializers.SerializerMethodField()
30     owner_username = serializers.CharField(source='owner.username', read_only=True)
31     category_name = serializers.CharField(source='category.name', read_only=True)
32     item_reviews = serializers.SerializerMethodField()
33     owner_rating = serializers.DecimalField(
34         source='owner.average_rating',
35         max_digits=3,
36         decimal_places=2,
37         read_only=True
38     )
39     owner_reviews_count = serializers.IntegerField(
40         source='owner.reviews_count',
41         read_only=True
42     )
43     # Цена с наценкой 20% (для арендатора)
44     price_with_markup = serializers.SerializerMethodField()
45     markup_rate = serializers.SerializerMethodField()
46
47     class Meta:
48         model = Item
49         fields = '__all__'
50         read_only_fields = ['owner', 'average_rating', 'reviews_count', 'created_at']
51
52     def get_price_with_markup(self, obj):
53         """Цена для арендатора с наценкой 20% платформы."""
54         price = Decimal(str(obj.price_per_day))
55         price_with_markup = (price * (1 + MARKUP_RATE)).quantize(Decimal('0.01'), rounding=ROUND_HALF_UP)
56         return str(price_with_markup)
57

```

```

58 def get_markup_rate(self, obj):
59     """Процент наценки платформы (20%)."""
60     return str(MARKUP_RATE * 100)
61
62 def get_booked_ranges(self, obj):
63     active_bookings = obj.bookings.filter(status__in=['pending', '
        confirmed']).order_by('start_date')
64     return [
65         {
66             'start_date': booking.start_date.isoformat(),
67             'end_date': booking.end_date.isoformat(),
68         }
69         for booking in active_bookings
70     ]
71
72 def get_item_reviews(self, obj):
73     from bookings.models import Review
74
75     reviews = Review.objects.filter(
76         booking__item=obj
77     ).filter(
78         is_hidden=False
79     ).select_related('booking__renter').order_by('-created_at')[:6]
80
81     return [
82         {
83             'id': review.id,
84             'rating': review.rating,
85             'comment': review.comment,
86             'created_at': review.created_at.isoformat(),
87             'author_username': review.booking.renter.username,
88         }
89         for review in reviews
90     ]
91
92
93 class ModerationUserSerializer(serializers.Serializer):
94     id = serializers.IntegerField(read_only=True)
95     username = serializers.CharField(read_only=True)
96     email = serializers.EmailField(read_only=True)
97     phone = serializers.CharField(read_only=True)
98     user_type = serializers.CharField(read_only=True)
99     full_name = serializers.CharField(read_only=True)
100     entrepreneur_name = serializers.CharField(read_only=True)
101     company_name = serializers.CharField(read_only=True)
102     passport_series = serializers.CharField(read_only=True)
103     passport_number = serializers.CharField(read_only=True)
104     inn = serializers.CharField(read_only=True)
105     kpp = serializers.CharField(read_only=True)
106     ogrnip = serializers.CharField(read_only=True)
107     profile_completed = serializers.BooleanField(read_only=True)
108
109
110 class ItemModerationRequestSerializer(serializers.ModelSerializer):

```

```

111 item = ItemSerializer(read_only=True)
112 submitted_by = ModerationUserSerializer(read_only=True)
113 reviewed_by_username = serializers.CharField(source='reviewed_by.username
    ', read_only=True)
114
115 class Meta:
116     model = ItemModerationRequest
117     fields = [
118         'id',
119         'item',
120         'submitted_by',
121         'action',
122         'status',
123         'item_snapshot',
124         'user_snapshot',
125         'rejection_reason',
126         'reviewed_by',
127         'reviewed_by_username',
128         'reviewed_at',
129         'created_at',
130         'updated_at',
131     ]
132     read_only_fields = fields
133
134
135 class ItemReviewSerializer(serializers.ModelSerializer):
136     author_username = serializers.CharField(source='author.username',
        read_only=True)
137
138 class Meta:
139     model = ItemReview
140     fields = ['id', 'item', 'author', 'author_username', 'rating', '
        comment', 'created_at']
141     read_only_fields = ['id', 'item', 'author', 'author_username', '
        created_at']

```

Автор ВКР

(подпись, дата)

В. А. Ермаков

Руководитель ВКР

(подпись, дата)

Е. А. Петрик

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

Место для диска